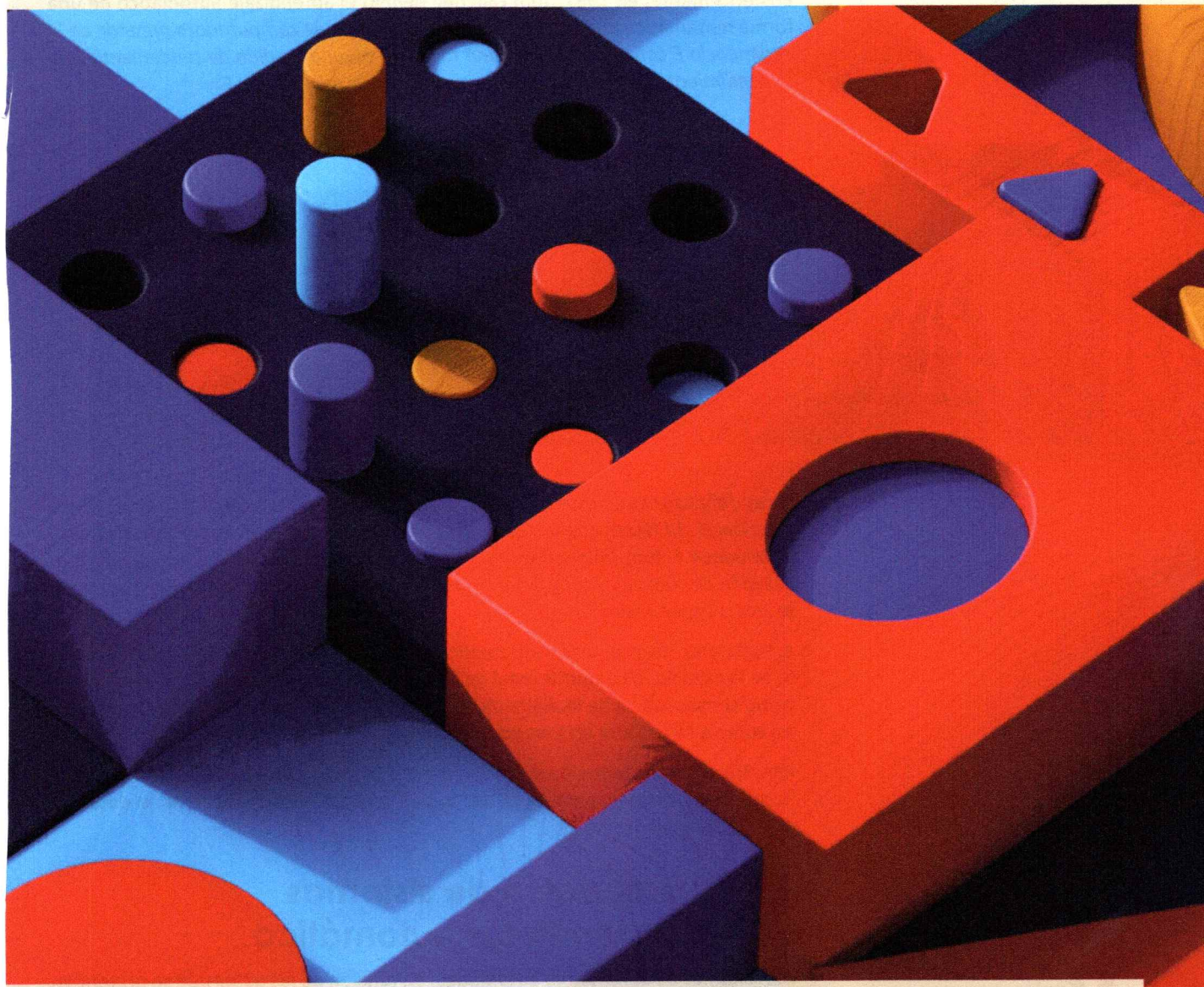
Unidad 2 Caracterización de sistemas de aprendizaje automático



En esta unidad veremos:

2.1 Introducción

2.2 Clasificación de sistemas de aprendizaje automático

2.3 Principales técnicas para desarrollar el aprendizaje automático

2.4 Algoritmos o modelos aplicados al aprendizaje automático

2.5 Procedimientos del *machine learning*: datos, identificación de patrones y toma de decisiones

2.6 Herramientas de aprendizaje automático

2.7 Aplicaciones del *machine learning*

2.1 Introducción

El concepto de **aprendizaje automático** o *machine learning* fue definido en 1959 por Arthur Samuel como *«el campo de estudio que da a las computadoras la capacidad de aprender sin ser programadas explícitamente»*.

Posteriormente, en 1997, Tom Mitchell definió el aprendizaje automático de una forma matemática al considerar que *«un programa de computadora aprende de la experiencia E con respecto a alguna tarea T y alguna medida de rendimiento R, si su rendimiento en T, medido por R, mejora con la experiencia E»*.

Ejemplo 1

La experiencia, la tarea y el rendimiento de un programa informático de un juego de ajedrez son los siguientes:

- E = la experiencia adquirida del juego de muchas partidas de ajedrez.
- T = la tarea de jugar una partida de ajedrez.
- R = la probabilidad de ganar la siguiente partida por parte del programa informático.

Otras definiciones de este concepto son las que han establecido Andrew Ng como *«la ciencia de hacer que las computadoras actúen sin estar explícitamente programadas»* o Tom Mitchell en uno de sus libros, *«el campo que se ocupa de la cuestión de cómo construir programas informáticos que mejoren automáticamente con la experiencia»*.

Actualmente el aprendizaje automático tiene un gran desarrollo debido al aumento de la capacidad computacional de los ordenadores y a la revolución de los datos como consecuencia de la digitalización y su acceso al registro de datos históricos, así como a su captura en tiempo real.

Permite realizar predicciones futuras a partir de conjuntos de datos existentes y hacer clasificaciones, para así poder tomar decisiones objetivas para un determinado problema.

2.2 Clasificación de sistemas de aprendizaje automático

Vocabulario

Los **datos de entrenamiento** son pares de datos formados por entradas y salidas conocidas.

Los sistemas de aprendizaje automático, en función de los datos que se pretendan analizar, se clasifican en: aprendizaje supervisado, no supervisado, semisupervisado y por refuerzo.

El **aprendizaje automático supervisado** parte de un conjunto de datos de entrenamiento que están previamente etiquetados, en el cual se conoce el valor del atributo objetivo o respuesta.

Esto hace que el algoritmo pueda aprender y obtener una función que sea capaz de generalizar y predecir la respuesta para un conjunto de valores nuevo.

Existen dos grandes grupos de métodos supervisados:

- **Algoritmos de regresión:** se utilizan para predecir un atributo numérico (Figura 2.1).
- **Algoritmos de clasificación:** se emplean para predecir un atributo categórico (Figura 2.2).

Ejemplo 2

En la Figura 2.1 se muestra un problema de aprendizaje supervisado de regresión, ya que son conocidos los valores del alquiler de algunas viviendas en función de los metros cuadrados de superficie.

A partir de estos datos se puede predecir cuál podría ser el valor del alquiler para una vivienda dada su superficie.

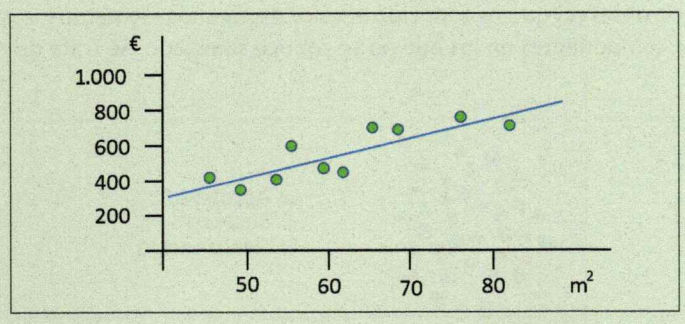


Figura 2.1 Aprendizaje supervisado. Problema de regresión.

Ejemplo 3

En la Figura 2.2 se observa que se conoce el valor de todos los elementos, suspenso o aprobado, por tanto, se trata de un sistema de aprendizaje supervisado. Como el atributo respuesta es categórico, es un problema de clasificación.

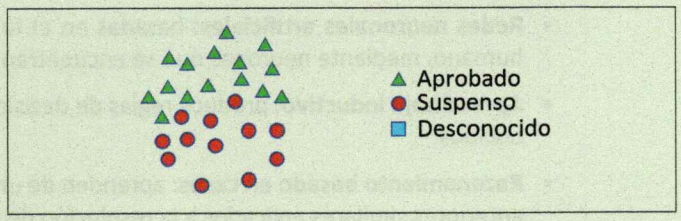


Figura 2.2 Aprendizaje supervisado. Problema de clasificación.

El **aprendizaje automático no supervisado** parte de un conjunto de datos sin etiquetar, siendo su objetivo el de agrupamiento, por lo que el algoritmo hace grupos de elementos similares por sí solo, sin ningún tipo de conocimiento que le permita testear el aprendizaje ni tener la capacidad de definir la individualidad de los integrantes de cada conjunto (Figura 2.3). El modelo identifica patrones que le permitan separar en grupos en función de sus atributos. Estas técnicas se conocen como *clustering*.

Ejemplo 4

En la Figura 2.3 están mezcladas diversas figuras. El algoritmo detecta elementos similares y los clasifica formando cuatro conjuntos, atendiendo a su forma.

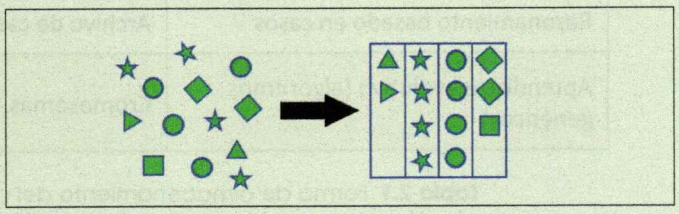


Figura 2.3 Aprendizaje no supervisado. Problema de clasificación.

El **aprendizaje semisupervisado** es un caso mixto de los vistos anteriormente. En esta ocasión, en el conjunto de datos de partida existen elementos etiquetados y otros que no lo están (Figura 2.4).

Ejemplo 5

En la Figura 2.4 se puede observar que es conocido el valor de algunos elementos, suspenso o aprobado, sin embargo, existen otros componentes en los que no se conoce su estado. Se trata de un sistema de aprendizaje semisupervisado.

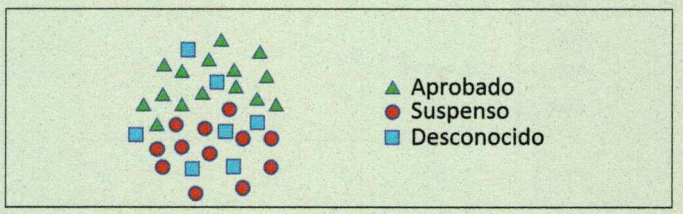


Figura 2.4 Aprendizaje semisupervisado.

2.3 Principales técnicas para desarrollar el aprendizaje automático

Las principales técnicas para el desarrollar el aprendizaje automático son las siguientes:

- **Redes neuronales artificiales**: basadas en el funcionamiento del cerebro humano, mediante neuronas que se encuentran interconectadas entre sí.
- **Aprendizaje inductivo**: produce reglas de decisión a partir de ejemplos conocidos.
- **Razonamiento basado en casos**: aprenden de una base de datos con casos anteriores similares aplicados a la resolución de problemas nuevos.
- **Aprendizaje evolutivo**: se fundamentan en el comportamiento de la naturaleza y su evolución y se implementan mediante algoritmos genéticos.

La principal diferencia entre estas cuatro técnicas es la forma en la que se almacena el conocimiento (Tabla 2.1).

Técnicas para desarrollar el aprendizaje automático	Forma de almacenamiento
Redes neuronales artificiales	Conjunto de pesos y umbrales que conectan cada neurona
Aprendizaje inductivo	Ramas de un árbol de decisión o conjunto de reglas
Razonamiento basado en casos	Archivo de casos
Aprendizaje evolutivo (algoritmos genéticos)	Cromosomas

Tabla 2.1 Forma de almacenamiento del conocimiento en las técnicas para desarrollar el aprendizaje automático.

2.3.1 Redes neuronales

Las redes neuronales artificiales (RNA) están inspiradas en el funcionamiento y en la estructura de las neuronas biológicas del cerebro.

Una neurona está formada por tres partes principalmente. En la Figura 2.5 se muestra un esquema de una neurona y se indican sus partes más importantes:

- Un **cuerpo celular** o **soma:** que procesa las señales que le llegan en forma de impulso nervioso de otras células y genera uno nuevo.
- Las **dendritas:** ramificaciones terminales que ejercen como canales de entrada de los impulsos en la neurona.
- El **axón**: prolongación del cuerpo celular que hace de canal transmisor de los impulsos que se generan, ramificándose en su extremo en forma de terminaciones.

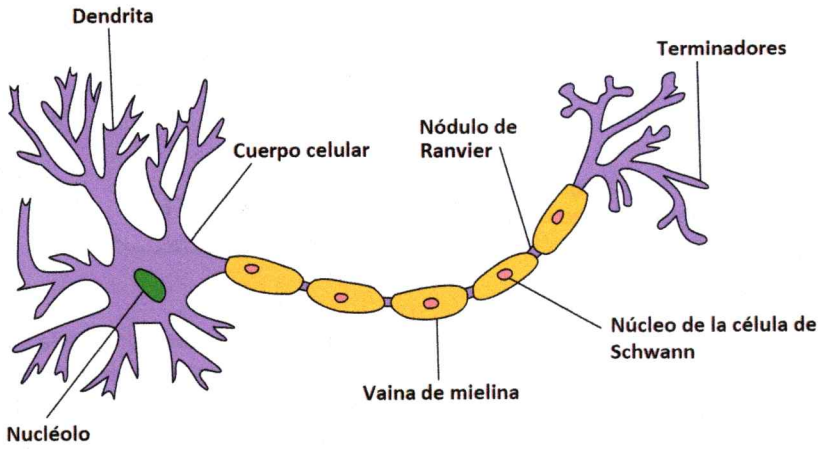


Figura 2.5
Esquema de una neurona biológica.

Las redes neuronales artificiales están constituidas por conjuntos de neuronas distribuidas en varias capas enlazadas entre sí. Cada neurona procesa la información que recibe y la propaga a través de toda la red.

Existen diversos tipos de redes neuronales, siendo las más comunes las que presentan una arquitectura de tres capas de neuronas, como puede observarse en la Figura 2.6:

- Una **capa de entrada** encargada de introducir a la red la información procedente del exterior.
- Una o varias **capas intermedias ocultas** situadas entre las capas de entrada y de salida, cuya misión es la de procesar la información recibida.
- Una **capa de salida** que suministra los resultados.

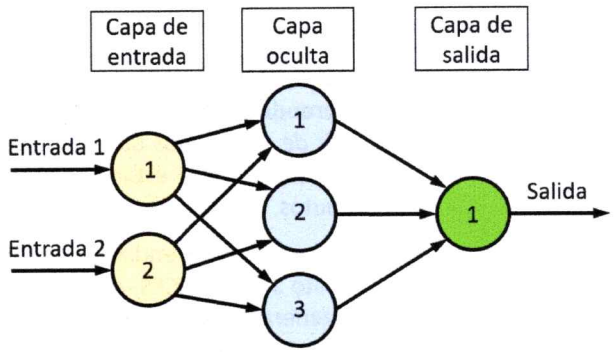


Figura 2.6
Capas de una red neuronal artificial.

El funcionamiento de una red neuronal artificial es, en términos generales, como se describe a continuación: la red recibe los valores que provienen de los datos de

entrada, llegando a las neuronas de la capa de entrada. Cada una de estas neuronas tiene asociado un peso con el que modifica el valor que ha recibido. A la salida de la neurona puede existir una función de activación que marca un umbral que permite continuar o impedir el paso a la siguiente neurona de la red. Los valores que se han calculado pasan a las neuronas de las siguientes capas de la red que vuelven a modificar los valores que reciben con los pesos y funciones de activación asociados hasta llegar a la capa de salida que proporciona el resultado final. En la Figura 2.7 se muestra el esquema de una neurona artificial.

La red será tanto más compleja cuanto mayor sea el número de capas ocultas, pero también será capaz de realizar funciones más complejas.

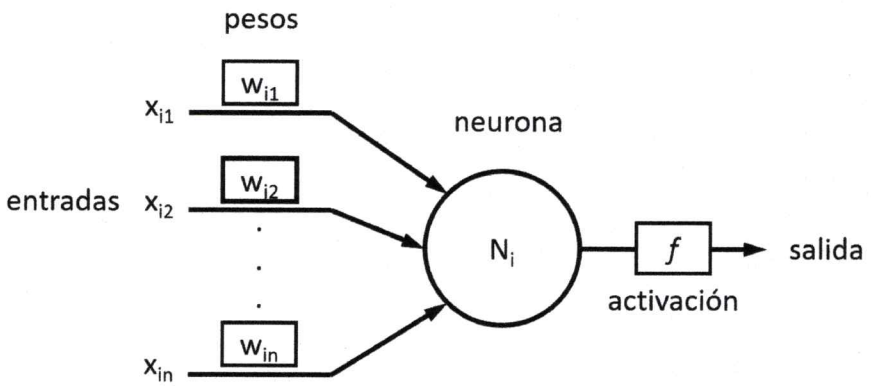


Figura 2.7
Esquema de una neurona artificial.

Se denominan **redes alimentadas hacia adelante** o *feedforward* a *aquellas redes en las que las neuronas no establecen conexiones con las neuronas de las capas anteriores, de la misma capa o consigo mismas.* Sin embargo, cuando existe alguno de estos tipos de conexión indicados, se llaman **redes realimentadas**, recurrentes o *feedback*.

La gran desventaja que presentan las redes neuronales frente a otros métodos es que la etapa de entramiento suele ser muy lenta. No obstante, una vez que ha finalizado dicha etapa y la red está entrenada, el conocimiento está repartido por toda la red y la velocidad de respuesta es inmediata.

Para recordar...

Las redes neuronales artificiales se utilizan en una gran variedad de problemas como, por ejemplo, reconocimiento por voz, sistemas de visión y robots autónomos, que son tareas complejas para resolver utilizando la programación basada en reglas.

En la Unidad 5 se tratará más detenidamente el funcionamiento de una red neuronal, así como su empleo en el aprendizaje profundo.

2.3.2 Aprendizaje inductivo

El **aprendizaje inductivo** consiste en la *estimación de una función desconocida a partir de unos datos de ejemplo que son conocidos.* Se representa en una tabla en la que las filas están formadas por dichos ejemplos y las columnas por los atributos.

El algoritmo de aprendizaje inductivo produce reglas de clasificación que tienen un formato SI-ENTONCES, desplegándose de forma iterativa para obtener conclusiones generales tomando como referencia los ejemplos conocidos.

Este tipo de problemas utiliza algoritmos que tienen la forma de un árbol de decisión y emplean conjuntos de reglas.

Para recordar...

Son aplicados en múltiples tareas de clasificación como, por ejemplo: decidir si a un cliente de un banco se le va a conceder un préstamo, la predicción de un fallo en la calidad de un producto o realizar el diagnóstico de una enfermedad.

Para la resolución de los problemas que se plantean, los métodos de aprendizaje inductivo utilizan, generalmente, modelos de árboles de decisión o también modelos de aprendizaje basados en reglas que permiten desplegar los conceptos a partir de la generalización del problema y descartando, precisamente, los casos que han permitido desarrollar esta generalización. En cambio, los sistemas de razonamiento basados en casos, que se muestran en el apartado siguiente, van a basarse en los ejemplos específicos que tienen almacenados en su base de datos y que va aumentando con el tiempo al incorporar nuevos problemas resueltos.

2.3.3 Razonamiento basado en casos

El **razonamiento basado en casos** (CBR), o *Case-Based Reasoning*, es un método de *machine learning* utilizado para determinar la solución de nuevos problemas empleando una base de datos que conserva instancias o casos anteriores previamente resueltos.

Las etapas de problema a resolver mediante un sistema de razonamiento en casos son las siguientes:

- Introducción en el sistema las características del problema que se desea resolver, valorando previamente cuáles de ellas son relevantes.
- Búsqueda en la base de datos de casos similares al planteado.
- Recuperación de los casos que se han encontrado en la búsqueda realizada.
- Valoración de la solución propuesta y si esta se adapta al nuevo problema que se plantea.
- En el caso de resolver el problema propuesto, se incorpora a la base de datos para poder aplicarlo en el futuro, cuando se plantee otro problema similar.

Los sistemas de CBR utilizan la experiencia para aprender de los éxitos que han tenido anteriormente y así poder evitar los fracasos que se han obtenido en otras ocasiones.

En el método de CBR los registros que están almacenados previamente en la base de datos se denominan **casos**, las características del problema se designan como **atributos** y la solución se llama **clase**.

El modelo más utilizado en el razonamiento basado en casos es el de los vecinos más cercanos, *k*-NN, que se desarrolla más adelante, en el apartado de *Modelos de clasificación aplicados al aprendizaje automático.*

Para recordar...

Los sistemas de CRB se utilizan para resolver problemas como el control de los equipos de los sistemas empleados en fabricación flexible, la predicción de fallos en máquinas productivas, la planificación de programas de trabajo o de viajes, el estudio de los costes que va a implicar un proyecto, el análisis de riesgos que supondría una determinada operación bancaria o en diagnósticos médicos.

2.3.4 Aprendizaje evolutivo

El **aprendizaje inductivo** está planteado con una concepción diferente al resto de modelos de aprendizaje automático ya que utiliza los **algoritmos genéticos**.

Estos algoritmos están inspirados en el comportamiento de la naturaleza, las leyes de la evolución biológica y la selección, por tanto, comienzan con un conjunto de soluciones de partida y mediante una simulación de un proceso evolutivo, estas soluciones iniciales van evolucionando y transformándose para adaptarse al entorno. De tal manera que en los algoritmos genéticos se acaban generando miles de trozos de código que se prueban para ver su rendimiento y se acaban eligiendo aquellos que proporcionan un resultado mejor, que son reproducidos en una generación posterior, se recombinan simulando las mutaciones y evolucionan obteniendo mejores fragmentos de código con el paso de las generaciones, de una forma similar a la evolución natural de las especies.

2.4 Algoritmos o modelos aplicados al aprendizaje automático

En este apartado se van a mostrar los algoritmos más comunes que se utilizan en clasificación, regresión, detección de anomalías, agrupamiento o *clustering*, refuerzo por aprendizaje y otros algoritmos como los empleados para la reducción de la dimensionalidad o el aprendizaje profundo.

No obstante, los algoritmos que se emplean para una tarea pueden adaptarse para realizar otras tareas. Por ejemplo, las redes neuronales artificiales o las máquinas de vector soporte pueden emplearse para clasificación y para regresión.

2.4.1 Algoritmos de clasificación

Los algoritmos de clasificación se utilizan cuando el resultado esperado es una etiqueta discreta en forma de conjunto finito de resultados. En el caso de dos posibilidades se trata de clasificación binaria; y si se quiere predecir más de dos categorías, se trata de clasificación multicategoría.

Ejemplo 6

Supongamos que una persona tiene un base de datos con todas las películas que ha visto y que a su vez ha etiquetado las que le gustan y las que no. Con esta información se puede predecir si una película que está actualmente en el cine le gustará. La solución a este problema será de tipo clasificación.

Los algoritmos de clasificación más empleados que se desarrollan en esta Unidad son el análisis discriminante, los *k* vecinos más próximos, los árboles de decisión como CART, C5.0 y *Random Forest*, el clasificador bayesiano ingenuo o *Naïve-Bayes*, la regresión logística, las máquinas de soporte vector y las redes neuronales artificiales.

Análisis discriminante

El **análisis discriminante lineal,** *linear dicriminant analysis* (LDA), es una técnica de análisis multivariante de clasificación supervisada utilizada para predecir a qué grupo se va a incorporar un elemento nuevo, cuya pertenencia se desconoce, a partir de otros conjuntos previamente conocidos. Calcula unas funciones lineales a partir de los atributos de su perfil, donde la función que alcanza mayor valor define el grupo al que pertenece el nuevo elemento de forma más probable. Cada elemento solamente puede pertenecer a un único grupo (Figura 2.8).

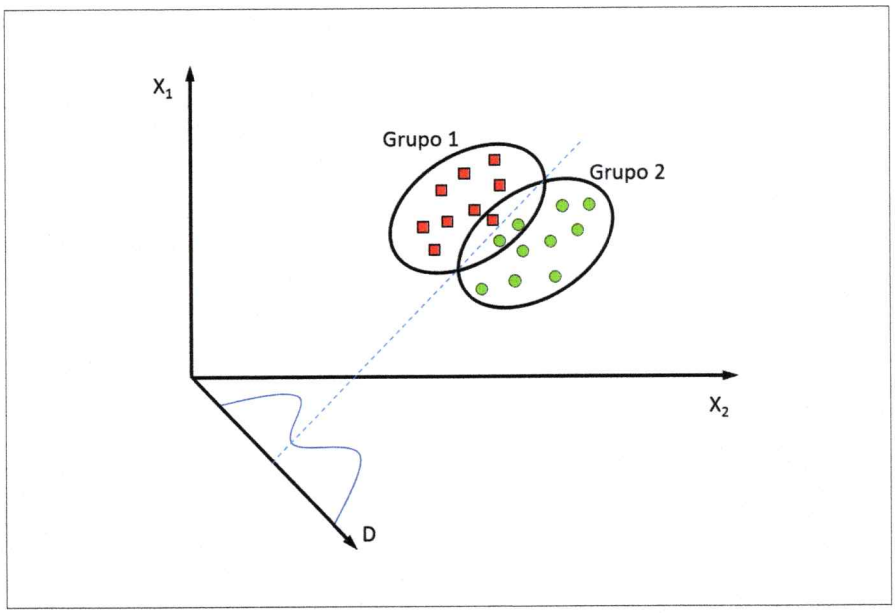


Figura 2.8
Análisis discriminante.

Ejemplo 7

El conjunto de datos de iris contiene datos sobre la longitud del sépalo de 150 flores para cada una de las especies de iris, que son versicolor, setosa y virgínica. Se separa el conjunto de datos en dos grupos, uno de entrenamiento con el 80% de los datos y otro con el 20 % restante sobre el que se realiza una predicción acerca del tipo de especie mediante el algoritmo de análisis discriminante, obteniendo la probabilidad de pertenencia a cada uno de los tres grupos y el porcentaje de clasificación correcta.

En la Figura 2.9 se muestra el código en R para este problema.

```
> library(MASS)
> data(iris)
> # Conjuntos de entrenamiento y validación
> entrenamiento=sample(seq(length(iris$Species)),length(iris$Species)*0.80,replace=FALSE)
> # Análisis dicriminante lineal
> lda.tr=lda(Species[entrenamiento]~.,data=iris[entrenamiento,])
> # Predicción de valores
> prediccion=predict(lda.tr,newdata=iris[-entrenamiento,],type="prob")
> data.frame(prediccion)[1:5,]
        class posterior.setosa posterior.versicolor posterior.virginica       x.LD1       x.LD2
6      setosa        0.4011521            0.2418214           0.3570265  0.73857766 -1.36661505
11     setosa        0.3976281            0.3348982           0.2674737  0.04764499 -1.03679633
12 versicolor        0.3264620            0.4144246           0.2591135 -0.20048782 -0.04485057
13 versicolor        0.3121991            0.5564092           0.1313916 -1.31415567 -0.11470406
19     setosa        0.4147416            0.2952449           0.2900136  0.27489636 -1.32035438
> # Cálculo del porcentaje de clasificación correcta
> 100*mean(prediccion$class==iris$Species[-entrenamiento])
[1] 60
```

Figura 2.9 Análisis discriminante de la longitud del sépalo.

K vecinos más próximos

El método de ***k* vecinos más próximos,** más conocido como *k-nearest neighbors* (*k*-NN), es un algoritmo supervisado no paramétrico empleado para tareas de clasificación basado en casos parecidos sin que sea necesario que haya una concordancia exacta con los datos del conjunto de entrenamiento, de forma que el algoritmo clasifica el nuevo elemento en el conjunto que le corresponde buscando en las observaciones más cercanas. Para ello, calcula las distancias a los elementos y las ordena de menor a mayor para asignar el grupo que tenga una frecuencia más alta con las distancias menores. No precisa construir un modelo

global, ya que asume que la clase de un elemento únicamente depende de los *k* vecinos más próximos.

Este método es muy sensible al parámetro *k*, que indica el número de vecinos seleccionado y la métrica de distancia utilizada. El valor de *k* se fija tras realizar varias pruebas o mediante una validación cruzada.

Como mayor inconveniente, este método necesita todo el conjunto de datos de entrenamiento para cada elemento nuevo a clasificar, por lo que requiere gran cantidad de memoria y consumo de procesamiento. Por ese motivo, es recomendable para conjuntos pequeños.

Ejemplo 8

Se considera un conjunto de datos formado por dos clases: círculos y triángulos (Figura 2.10), que será el conjunto de datos de entrenamiento. Se quiere predecir a qué clase pertenece el nuevo elemento.

Se calcula la distancia entre este elemento y el resto de los datos. Aunque se podría hacer con todos, se consideran los $k = 4$ vecinos más próximos y se observa que hay más triángulos que círculos, por tanto, se puede predecir que el nuevo elemento será un triángulo.

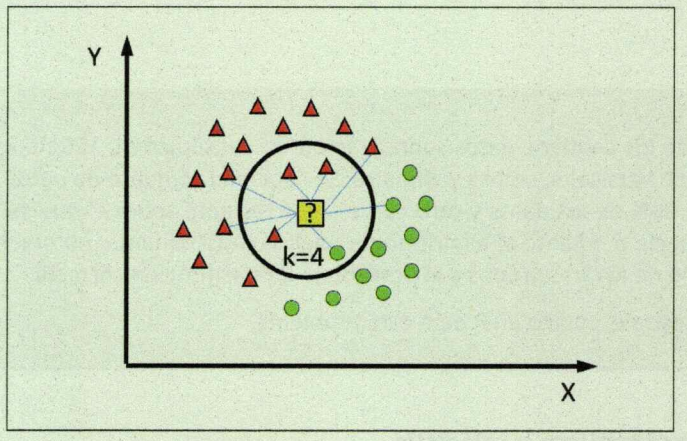


Figura 2.10 *K* vecinos más próximos.

Los pasos básicos del algoritmo son los siguientes:

- Calcular la distancia entre el elemento que se quiere clasificar y el resto de datos del conjunto de entrenamiento.
- Encontrar los *k* vecinos más cercanos, es decir, que tengan la menor distancia, según la métrica empleada.
- Realizar la votación de las etiquetas, que decidirán el resultado de clasificación final.

Ejemplo 9

Se considera el conjunto de datos Aid2, que contiene datos sobre pacientes con sida y se realiza un problema de clasificación utilizando el 80 % de los datos como conjunto de entrenamiento y $k = 5$ vecinos más próximos, para realizar la predicción sobre el 20% restante de datos y comprobar el porcentaje de clasificación correcta, como se muestra en la Figura 2.11.

```
> library(MASS)
> data(Aids2)
> library(class)
> set.seed(123)
> # Selección de las variables
> x=data.frame(Aids2[,3],Aids2[,7])
> str(x)
'data.frame':   2843 obs. of  2 variables:
 $ Aids2...3.: int  10905 11029 9551 9577 10015 9971 10746 10042 10464 10439 ...
 $ Aids2...7.: int  35 53 42 44 39 36 36 31 26 27 ...
> # Conjunto de entrenamiento
> entrenamiento=sample(seq(length(Aids2$status)),length(Aids2$status)*0.80,replace=FALSE)
> # K-vecinos más próximos (k-NN)
> prediccion=knn(x[entrenamiento,],x[-entrenamiento,],Aids2$status[entrenamiento],k=5,prob=TRUE)
> table(prediccion,Aids2$status[-entrenamiento])

prediccion   A   D
         A 152  52
         D  75 290
> # Cálculo del porcentaje de clasificación correcta
> 100*mean(prediccion==Aids2$status[-entrenamiento])
[1] 77.68014
```

Figura 2.11 Clasificación con algoritmo *k*-NN.

Árboles de decisión

Los **árboles de decisión** son algoritmos para hacer clasificaciones realizando particiones sucesivas, mediante la técnica denominada *segmentación jerárquica,* agrupando observaciones similares. Analizan situaciones que presentan varias posibilidades de decisión y toman la que consideran mejor.

El método, cuando se encuentra ante una decisión, utiliza la información del conjunto de datos de entrenamiento para buscar una correlación entre el elemento nuevo y los conocidos y así poder elegir la opción correcta. También son empleados para realizar tareas de regresión.

Los árboles de decisión están formados por los siguientes componentes:

- Los **nodos:** son las variables de entrada y plantean las opciones de decisión, siendo el primer elemento del árbol el nodo raíz.
- Las **ramas**: indican los valores que pueden tomar las variables de entrada y unen los nodos entre sí.
- Las **hojas**: muestran los valores de las variables de salida, es decir, el resultado de la decisión.

Para recordar...

La profundidad de un árbol es el número máximo de nodos en una rama.

Ejemplo 10

Una aplicación práctica de los árboles de decisión son las partidas de ajedrez, donde la máquina tiene muchas posibilidades de movimientos, pero ha de tomar la decisión de elegir aquel que tiene una mayor probabilidad de éxito, teniendo en cuenta los posibles movimientos del adversario y reconfigurando los cálculos cada vez que se realizan movimientos, hasta llegar a la hoja terminal, que sería la victoria.

Algunos de los algoritmos de árboles de decisión más utilizados son los siguientes:

- El algoritmo de **árboles de clasificación y regresión** (CART) genera árboles de decisión binarios, por lo que cada nodo se divide en dos ramas.

- Los algoritmos **C5.0**, que es una evolución del **C4.5** que a su vez lo es de **ID3**, forman una familia de algoritmos de los más utilizados en tareas de clasificación.
- El algoritmo de **bosque aleatorio** o *Random Forest* (RF) combina varios árboles y la salida de cada uno de ellos proporciona un voto, siendo la opción más votada la elegida como la respuesta del modelo. El funcionamiento es como sigue:
  - Se seleccionan *k* características del conjunto de datos y se crea un árbol de decisión con las *k* columnas.
  - Se crean *n* árboles en la que se varía la cantidad *k* de características.
  - Para cada uno de los árboles se obtiene una clasificación del elemento buscado.
  - Se calcula el voto de cada clase que proporciona cada uno de los árboles creados y se selecciona la más votada.

Este método no es apropiado para conjuntos de datos pequeños y requiere mucho tiempo durante la fase de entrenamiento.

Ejemplo 11

En la Figura 2.12 se muestran un ejemplo del problema resuelto con el algoritmo CART para el conjunto de datos iris, con un conjunto de datos de entrenamiento del 70 % y el resto para realizar una predicción acerca del tipo de especie según la longitud del pétalo (petal.lenght), del sépalo (sepal.lenght) y el ancho del pétalo (petal.width).

```
> library(MASS)
> data(iris)
> library(tree)
> set.seed(123)
> # Conjunto de entrenamiento
> entrenamiento=sample(seq(length(iris$Species)),
+                      length(iris$Species)*0.70,replace=FALSE)
> iris.tree = tree(iris$Species~.,iris,subset=entrenamiento)
> summary(iris.tree)

Classification tree:
tree(formula = iris$Species ~ ., data = iris, subset = entrenamiento)
Variables actually used in tree construction:
[1] "Petal.Length" "Petal.Width"  "Sepal.Width"
Number of terminal nodes:  6
Residual mean deviance:  0.186 = 18.42 / 99
Misclassification error rate: 0.0381 = 4 / 105
> # Dibujo del árbol
> plot(iris.tree);text(iris.tree,pretty=0)
> iris.tree
node), split, n, deviance, yval, (yprob)
      * denotes terminal node

 1) root 105 230.300 virginica ( 0.34286 0.30476 0.35238 )
   2) Petal.Length < 2.45 36   0.000 setosa ( 1.00000 0.00000 0.00000 ) *
   3) Petal.Length > 2.45 69  95.290 virginica ( 0.00000 0.46377 0.53623 )
     6) Petal.Width < 1.75 35  24.880 versicolor ( 0.00000 0.88571 0.11429 )
      12) Petal.Width < 1.45 21   0.000 versicolor ( 0.00000 1.00000 0.00000 ) *
      13) Petal.Width > 1.45 14  16.750 versicolor ( 0.00000 0.71429 0.28571 )
        26) Sepal.Width < 2.85 5   6.730 virginica ( 0.00000 0.40000 0.60000 ) *
        27) Sepal.Width > 2.85 9   6.279 versicolor ( 0.00000 0.88889 0.11111 ) *
     7) Petal.Width > 1.75 34   9.023 virginica ( 0.00000 0.02941 0.97059 )
      14) Petal.Length < 4.95 6   5.407 virginica ( 0.00000 0.16667 0.83333 ) *
      15) Petal.Length > 4.95 28   0.000 virginica ( 0.00000 0.00000 1.00000 ) *
> # Predicción de valores
> prediccion <- predict(iris.tree,iris[-entrenamiento,],type="class")
> summary(prediccion)
    setosa versicolor  virginica
        14         17         14
> # Matriz de confusión
> tabla <-with(iris[-entrenamiento,],table(prediccion,Species))
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 93.33333
```

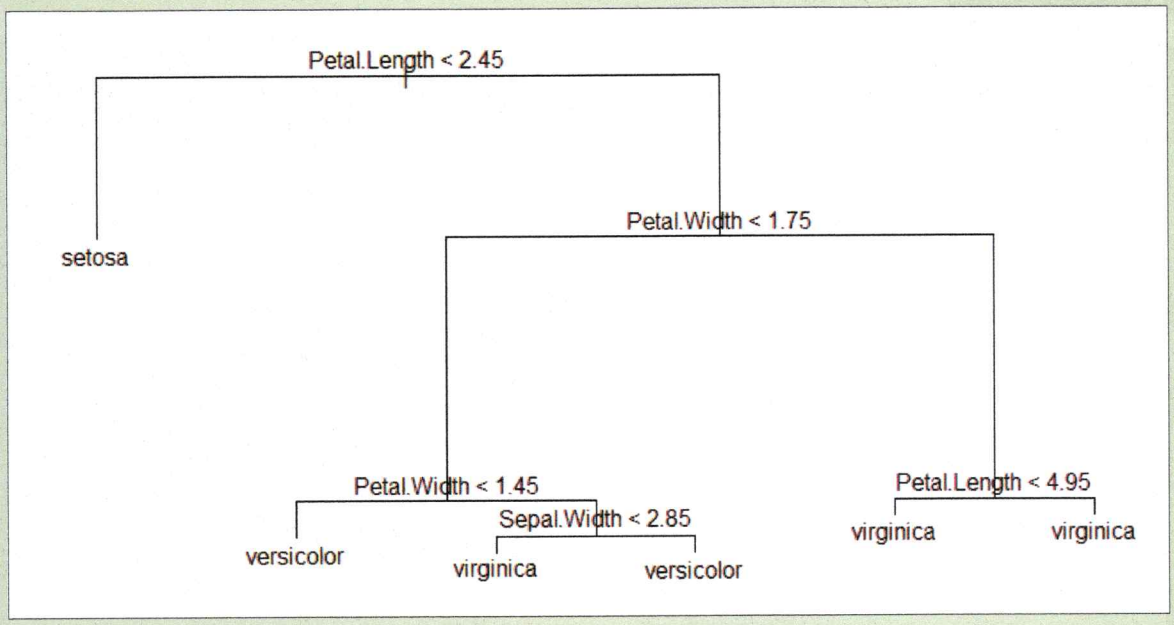


Figura 2.12 Clasificación con algoritmo CART.

Ejemplo 12

En la Figura 2.13 se realiza el mismo ejemplo anterior con el algoritmo C5.0.

```
> library(MASS)
> data(iris)
> library(C50)
> set.seed(123)
> # Conjuntos de entrenamiento y clasificación
> iris.indices <- sample(1:nrow(iris),size=105)
> iris.entrenamiento <- iris[iris.indices,]
> iris.test <- iris[-iris.indices,]
> # Modelo de entrenamiento
> modelo.C50 <- C5.0(Species~., data = iris.entrenamiento,rules=TRUE)
> modelo.C50

Call:
C5.0.formula(formula = Species ~ ., data = iris.entrenamiento, rules = TRUE)

Rule-Based Model
Number of samples: 105
Number of predictors: 4

Number of Rules: 4

Non-standard options: attempt to group attributes

> # Predicción de valores
> prediccion <- predict(modelo.C50,newdata=iris.test)
> # Matriz de confusión
> tabla <- table(prediccion, iris.test$Species)
> tabla

prediccion   setosa versicolor virginica
  setosa         14          0         0
  versicolor      0         17         0
  virginica       0          1        13
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 97.77778
```

Figura 2.13 Clasificación con algoritmo C5.0.

Ejemplo 13

En la Figura 2.14 se realiza nuevamente el mismo caso que en el Ejemplo 11 con el algoritmo *Random Forest*.

```
> library(MASS)
> data(iris)
> library(randomForest)
> set.seed(123)
> datos <- iris
> # Conjuntos de entrenamiento y clasificación
> tamano.total <- nrow(datos)
> tamano.entrenamiento <- round(tamano.total*0.7)
> iris.indices <- sample(1:tamano.total , size=tamano.entrenamiento)
> datos.entrenamiento <- datos[iris.indices,]
> datos.test <- datos[-iris.indices,]
> # Modelo de entrenamiento
> modelo.RF <- randomForest(Species~., data=datos.entrenamiento)
> modelo.RF

Call:
 randomForest(formula = Species ~ ., data = datos.entrenamiento)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 5.71%
Confusion matrix:
           setosa versicolor virginica class.error
setosa         36          0         0  0.00000000
versicolor      0         29         3  0.09375000
virginica       0          3        34  0.08108108
> # Predicción de valores
> predicciones <- predict(modelo.RF, datos.test)
> # Matriz de confusión
> tabla <- with(datos.test,table(predicciones, Species))
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 97.77778
```

Figura 2.14 Clasificación con algoritmo *Random Forest*.

Algoritmo clasificador bayesiano ingenuo

El algoritmo **clasificador bayesiano ingenuo**, más conocido como *Naïve Bayes*, es una técnica de clasificación probabilística que está fundamentada en el teorema de Bayes y está basada en la idea de que el mejor modelo es el más probable, asumiendo que la ocurrencia de una determinada característica es independiente de que sucedan las otras particularidades. Este es el motivo por el que se le llama ingenuo.

Asume tanto la independencia entre los atributos como que estos sean nominales. Solo requiere un pequeño conjunto de datos de entrenamiento para realizar la estimación de los parámetros necesarios: medias y varianzas.

Es un algoritmo simple y efectivo que se utiliza frecuentemente en la clasificación de texto.

Ejemplo 14

En la Figura 2.15 se muestra el código para clasificar las especies del conjunto de datos iris, con un conjunto de datos de entrenamiento del 80 % y el resto para realizar una predicción acerca del tipo de especie.

```
> library(MASS)
> data(iris)
> library(e1071)
> library(caret)
> set.seed(123)
> # Conjunto de entrenamiento
> entrenamiento<-createDataPartition(iris$Species,p=0.80,list=F)
> modelo.NB<-naiveBayes(Species~.,data=iris[entrenamiento,])
> print(modelo.NB)

Naive Bayes Classifier for Discrete Predictors

Call:
naiveBayes.default(x = X, y = Y, laplace = laplace)

A-priori probabilities:
Y
    setosa versicolor  virginica
 0.3333333  0.3333333  0.3333333

Conditional probabilities:
            Sepal.Length
Y               [,1]      [,2]
  setosa     4.9800 0.3567661
  versicolor 5.9400 0.4903165
  virginica  6.6375 0.6949221

            Sepal.Width
Y               [,1]      [,2]
  setosa     3.3700 0.3450752
  versicolor 2.7700 0.3267556
  virginica  3.0125 0.3123012

            Petal.Length
Y               [,1]      [,2]
  setosa     1.4650 0.1717930
  versicolor 4.2325 0.4676112
  virginica  5.6225 0.5775667

            Petal.Width
Y               [,1]      [,2]
  setosa     0.2400 0.0928191
  versicolor 1.3275 0.2087662
  virginica  2.0700 0.2662176

> # Predicción de valores
> pred<-predict(modelo.NB,iris[-entrenamiento,])
> # Matriz de confusión
> tabla=table(iris[-entrenamiento,]$Species,pred,dnn=c("Real","Predicion"))
> tabla
            Predicion
Real         setosa versicolor virginica
  setosa         10          0         0
  versicolor      0         10         0
  virginica       0          2         8
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 93.33333
```

Figura 2.15 Clasificación con algoritmo clasificador ingenuo.

Regresión logística

Se emplea para predecir el resultado de una variable dependiente categórica utilizando un conjunto dado de variables independientes. Modela las probabilidades de problemas de clasificación con dos resultados posibles. Por tanto, el resultado debe ser un valor categórico o discreto. Puede ser SÍ o NO, 0 o 1, verdadero o falso, etcétera, pero en lugar de dar el valor exacto como 0 y 1, asigna a un valor real otro valor probabilístico dentro de un rango entre 0 y 1.

Se representa mediante una curva con forma de S, tal y como se puede observar en la Figura 2.16, llamada función sigmoidea, que es una función matemática que se utiliza para asignar los valores predichos a las probabilidades.

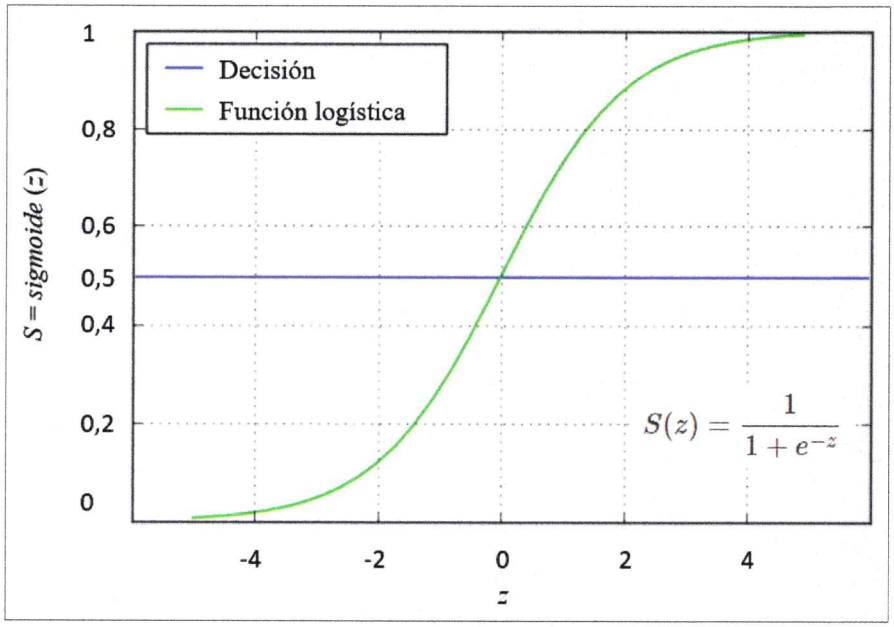


Figura 2.16
Función logística.

Ejemplo 15

En la Figura 2.17 se muestra el código de la regresión logística para clasificar la especie en función de la longitud del sépalo, con un subconjunto de las 100 primeras observaciones para las especies setosa y versicolor de los datos iris, con un conjunto de datos de entrenamiento del 80 %.

```
> library(MASS)
> library(caret)
> data(iris)
> library(MASS)
> library(caret)
> data(iris)
> set.seed(123)
> datos.iris<- iris
>
> # Clases setosa y versicolor
> datos.iris<-datos.iris[1:100,]
> # Conjuntos entrenamiento y validación
> indices<-sample(1:100,85)
> iris.entrenamiento<-datos.iris[indices,]
> iris.test<-datos.iris[-indices,]
>
> # Modelo de Regresión Logística
> y<-iris.entrenamiento$Species; x<-iris.entrenamiento$Sepal.Length
> modelo.RLOG<-glm(y~x, family = 'binomial')
> # Probabilidad de setosa o versicolor según longitud sépalo
> summary(modelo.RLOG)

Call:
glm(formula = y ~ x, family = "binomial")

Deviance Residuals:
     Min        1Q    Median        3Q       Max
-2.06255  -0.54330   0.02923   0.50381   2.19652

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -25.831      5.463  -4.728 2.27e-06 ***
x              4.798      1.017   4.719 2.37e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 117.823  on 84  degrees of freedom
Residual deviance:  58.295  on 83  degrees of freedom
AIC: 62.295
```

```
> # Predicciones
> validacion<- data.frame(x=iris.test$Sepal.Length)
> predicted_val<-predict(modelo.RLOG, validacion, type="response")
> prediccion<-data.frame(iris.test$Sepal.Length, iris.test$Species,
+                        predicted_val)
> prediccion
   iris.test.Sepal.Length iris.test.Species predicted_val
1                     5.1            setosa    0.20443712
2                     4.9            setosa    0.08960371
3                     4.9            setosa    0.08960371
4                     5.4            setosa    0.52017686
5                     5.2            setosa    0.29339696
6                     4.9            setosa    0.08960371
7                     5.5            setosa    0.63659094
8                     5.0            setosa    0.13721265
9                     6.4        versicolor    0.99245443
10                    6.5        versicolor    0.99531674
11                    5.7        versicolor    0.82058147
12                    5.8        versicolor    0.88081166
13                    6.1        versicolor    0.96892187
14                    6.3        versicolor    0.98786406
15                    6.2        versicolor    0.98053588
>
> # Matriz de confusión
> pred <- rep("Yes", nrow(iris.test))
> pred[predicted_val < 0.5] <- "No"
> tabla <- table(pred,iris.test$Species); tabla

pred  setosa versicolor virginica
  No       6          0         0
  Yes      2          7         0
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum((tabla[2,])) / sum(tabla)
[1] 60
```

Figura 2.17 Regresión logística.

Máquinas de vector soporte

Las **máquinas de vector soporte,** *support vector machines* (SVM), son un conjunto de algoritmos para clasificación en dos grupos. Separan un grupo dado de datos de entrenamiento etiquetados binarios mediante un hiperplano que está a una distancia máxima de ellos (conocido como el **hiperplano de margen máximo**). De esta forma, elementos que son etiquetados con una categoría estarán a un lado del hiperplano y los casos que se encuentren en la otra categoría se situarán al otro lado, como puede observarse en la Figura 2.18. Al vector formado por los puntos más cercanos al hiperplano se le llama **vector de soporte.**

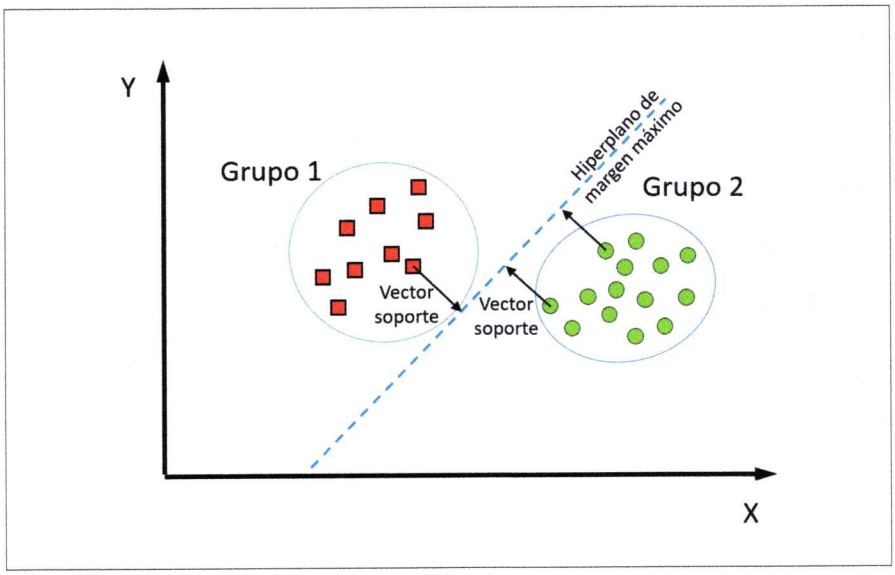


Figura 2.18
Máquinas de vector soporte.

Para los casos en los que no es posible una separación lineal, pueden trabajar en combinación con la técnica de *kernels* que realiza un mapeo no lineal a un espacio de dimensión superior. El hiperplano encontrado por la SVM en el espacio de características corresponde a un límite de decisión no lineal en el espacio de entrada. Hay que tener en cuenta que la técnica del *kernel* no forma parte de SVM. Se puede utilizar con otros clasificadores lineales.

El método se puede extender para resolver el problema de clasificación de datos en más de dos categorías, mediante dos procedimientos:

- **Uno contra el resto:** para *k* clases, se entrenan *k* clasificadores binarios. Cada uno determina si un ejemplo pertenece a su propia clase frente a cualquier otra. Se considera que el clasificador con la salida más grande es la clase del ejemplo.
- **Uno contra uno:** se entrena un clasificador binario para cada par de clases. Se utiliza un procedimiento de votación para combinar los resultados.

Ejemplo 16

En la Figura 2.19 se muestra el código para la clasificación del conjunto de datos iris, con un conjunto de datos de entrenamiento del 80 % y el resto para realizar una predicción acerca del tipo de especie.

```
> library(MASS)
> data(iris)
> library(e1071)
> set.seed(123)
> # Construcción de conjuntos de entrenamiento y validación
> filas <- c(1:nrow(iris))
> test.filas <- sample(filas, size = (length(filas)*0.20))
> entrenamiento <- iris[-test.filas ,]
> test <- iris[test.filas ,]
> modelo.SVM <- svm(Species ~ ., data = entrenamiento, kernel = 'linear')
> modelo.SVM

Call:
svm(formula = Species ~ ., data = entrenamiento, kernel = "linear")


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  1

Number of Support Vectors:  26

> # Matriz de confusión
> tabla<-table(prediccion = predict(modelo.SVM, entrenamiento),
+              real = entrenamiento$Species)
> tabla
            real
prediccion   setosa versicolor virginica
  setosa         40          0         0
  versicolor      0         38         1
  virginica       0          3        38
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 96.66667
```

Figura 2.19 Máquinas de vector soporte.

Redes neuronales artificiales

Las redes neuronales artificiales, vistas en el Apartado 2.3, pueden aplicarse para resolver problemas de clasificación.

Ejemplo 17

En la Figura 2.20 se muestra el código para clasificar las especies del conjunto de datos iris, con un conjunto de datos de entrenamiento del 70 % y el resto para realizar una predicción acerca del tipo de especie a partir de la longitud y anchura del sépalo y del pétalo, mediante una red neuronal con tres neuronas en la capa oculta.

```
> library(MASS)
> data(iris)
> set.seed(123)
> library(nnet)
> # Construcción de conjuntos de entrenamiento y validación
> tamano.total <- nrow(iris)
> tamano.entrenamiento <- round(tamano.total*0.7)
> iris.indices <- sample(1:tamano.total , size=tamano.entrenamiento)
> iris.entrenamiento <- iris[iris.indices,]
> iris.test <- iris[-iris.indices,]
> # Modelo de entrenamiento
> modelo.RNA <- nnet(Species ~ ., size=3,
+                    decay=0.1, trace=F, data=iris.entrenamiento)
> modelo.RNA
a 4-3-3 network with 27 weights
inputs: Sepal.Length Sepal.Width Petal.Length Petal.Width
output(s): Species
options were - softmax modelling  decay=0.1
> # Predicción de valores
> predicciones <- data.frame(predict(modelo.RNA, iris.test),
+                            Specie=predict(modelo.RNA,iris.test,
+                                           type="class"))
> # Matriz de confusión
> tabla <- table(predicciones$Specie,iris.test$Species,
+                dnn = c("predicción","real"))
> tabla
            real
predicción   setosa versicolor virginica
  setosa         14          0         0
  versicolor      0         17         0
  virginica       0          1        13
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 97.77778
```

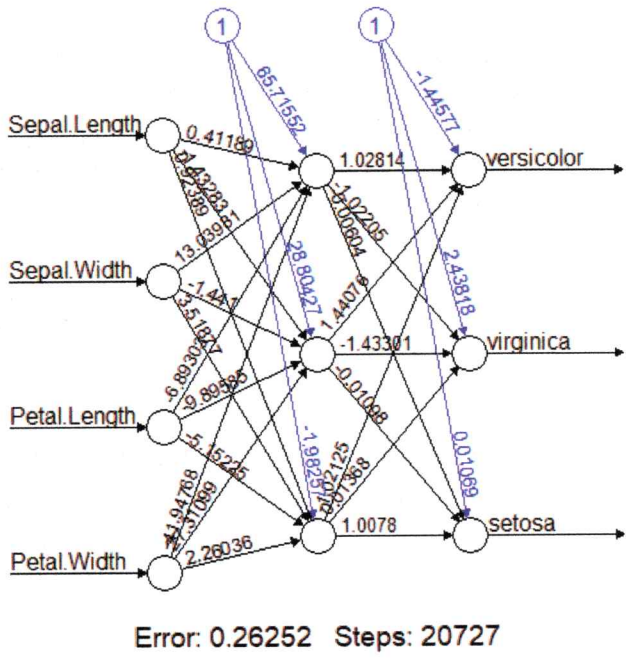


Figura 2.20 Clasificación con red neuronal artificial.

2.4.2 Algoritmos de regresión

Los algoritmos de regresión más empleados que se desarrollan en esta Unidad son la regresión lineal simple y múltiple y la regresión lineal múltiple paso a paso.

Regresión lineal simple y múltiple

La regresión lineal es una técnica que busca la relación lineal entre unas variables independientes de entrada y una variable dependiente de salida.

En el caso de la regresión lineal, siempre se tiene una única variable independiente, cuyo el objetivo es encontrar la recta que mejor ajuste a los valores de manera que el error entre los valores predichos y los valores reales sea mínimo (Figura 2.21).

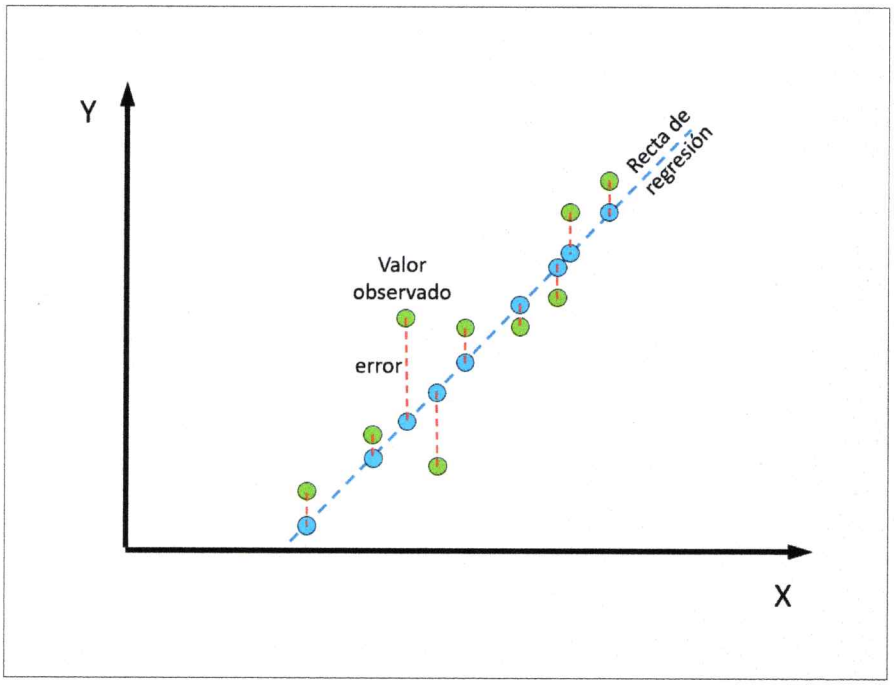


Figura 2.21
Recta de regresión lineal simple.

Loa valores diferentes que pueden tomar los coeficientes de regresión o pesos provocan rectas de regresión diferentes. Para calcular la de mejor ajuste, existen varios métodos para determinar dicha recta:

- **Método de mínimos cuadrados ordinarios:** que trata de minimizar la suma de los cuadrados de la diferencia entre el valor predicho y el valor observado para cada punto.
- **Método de máxima verosimilitud:** que busca maximizar la probabilidad.
- **Método del gradiente descendente:** que es un sistema de optimización iterativo que busca minimizar el error mediante el cálculo del gradiente de la función de coste.

Las hipótesis de la regresión lineal simple son las siguientes:

- Linealidad entre la variable independiente y dependiente.
- Homogeneidad, pues el valor medio del error debe ser cero.
- Homocedasticidad, es decir, la varianza de los errores tiene que ser constante.
- Independencia de las observaciones.
- Distribución normal de los residuos.

Ejemplo 18

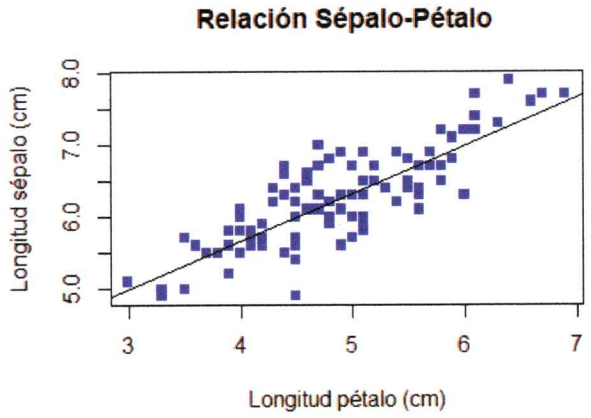
En la Figura 2.22 se muestra el código de regresión lineal simple entre las longitudes del pétalo y del sépalo de la especie setosa del conjunto de datos iris. Se eliminan las observaciones 51 a 150 que corresponden a las especies versicolor y virgínica y se comprueban las hipótesis del modelo. Asimismo, se hace una predicción para la longitud de pétalo de 5 cm y se calculan los intervalos de confianza de la predicción.

```
> library(MASS)
> data(iris)
> set.seed(123)
> library(nortest)
> library(lmtest)
> datos.iris <- iris
>
> # Eliminación especie setosa del estudio
> datos.iris<-datos.iris[51:150,]
> # Correlación entre las variables
> cor(datos.iris[,1:4])
             Sepal.Length Sepal.Width Petal.Length Petal.Width
Sepal.Length    1.0000000   0.5538548    0.8284787   0.5937094
Sepal.Width     0.5538548   1.0000000    0.5198023   0.5662025
Petal.Length    0.8284787   0.5198023    1.0000000   0.8233476
Petal.Width     0.5937094   0.5662025    0.8233476   1.0000000
> # Relacion entre longitud del pétalo y sépalo
> plot(y=datos.iris$Sepal.Length, x=datos.iris$Petal.Length ,
+      col="blue",
+      pch=15,
+      main="Relación Sépalo-Pétalo",
+      xlab="Longitud pétalo (cm)",
+      ylab="Longitud sépalo (cm)",)
> # Modelo de regresión lineal simple
> modelo.RLS<-lm(Sepal.Length~Petal.Length,datos.iris)
> modelo.RLS

Call:
lm(formula = Sepal.Length ~ Petal.Length, data = datos.iris)

Coefficients:
 (Intercept)  Petal.Length
      2.9987        0.6652

> abline(modelo.RLS)
```




```
> summary(modelo.RLS)

Call:
lm(formula = Sepal.Length ~ Petal.Length, data = datos.iris)

Residuals:
     Min       1Q   Median       3Q      Max
-1.09194 -0.26570  0.00761  0.21902  0.87502

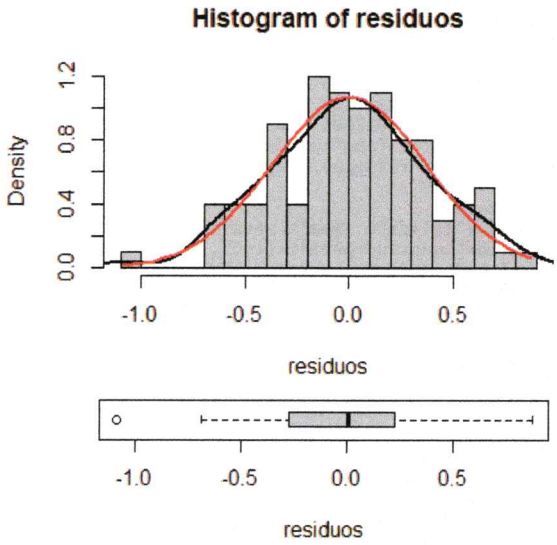
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   2.99871    0.22593   13.27   <2e-16 ***
Petal.Length  0.66516    0.04542   14.64   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3731 on 98 degrees of freedom
Multiple R-squared:  0.6864,    Adjusted R-squared:  0.6832
F-statistic: 214.5 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
> # Análisis de los residuos del modelo
> residuos=residuals(modelo.RLS)
> # Comprobación media de los residuos igual a cero
> mean(residuos)
[1] -3.057179e-17
> # Comprobación normalidad de los residuos
> par(mfrow=c(2,1))
> hist(residuos, breaks=24, freq= FALSE, col="gray")
> lines(density(residuos), lwd = 2, col = 'black')
> # Comparación con la curva normal
> x <- seq(min(residuos), max(residuos), length = 40)
> f <- dnorm(x, mean = mean(residuos), sd = sd(residuos))
> lines(x, f, col = "red", lwd = 2)
> boxplot(residuos, bty="l", range=1.5, col="gray",
+         horizontal=T,xlab="residuos")
> lillie.test(residuos)

        Lilliefors (Kolmogorov-Smirnov) normality test

data:  residuos
D = 0.043847, p-value = 0.9076
```

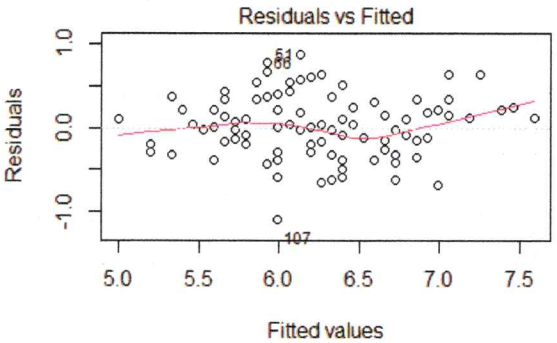



```
> # Comprobación homocedasticidad
> bptest(modelo.RLS)

        studentized Breusch-Pagan test

data:  modelo.RLS
BP = 0.04988, df = 1, p-value = 0.8233

> plot(modelo.RLS,1)
> # Comprobación covarianza término error y variable
> # explicativa es cero
> cov(residuos,datos.iris$Petal.Length)
[1] -3.589982e-17
> cor(residuos,datos.iris$Petal.Length)
[1] -1.171453e-16
```




```
> # Predicción para longitud pétalo = 5
> # Intervalos confianza de las predicciones
> newdatos <- data.frame(Petal.Length=5)
> predict(modelo.RLS,newdatos,interval="prediction",
+         type="response")
       fit      lwr      upr
1 6.324525 5.580399 7.068651
```

Figura 2.22 Regresión lineal simple.

El concepto de regresión lineal simple se amplía en la regresión lineal múltiple con la extensión a *k* variables independientes y sustituyendo la recta de regresión por un hiperplano de regresión.

Ejemplo 19

En la Figura 2.23 se muestra el código de regresión lineal múltiple de la longitud de pétalo a partir de la longitud y anchura del sépalo y de la anchura del pétalo, con un conjunto de datos de entrenamiento del 70 % y el resto para realizar una predicción de la longitud del pétalo. Se calcula el error RMSE.

```
> library(MASS)
> data(iris)
> set.seed(123)
> library(nortest)
> library(lmtest)
> library(Metrics)
> # Construcción conjuntos entrenamiento y validación
> tamano.total <- nrow(iris)
> tamano.entrenamiento <- round(tamano.total*0.7)
> iris.indices <- sample(1:tamano.total ,
+                        size=tamano.entrenamiento)
> iris.entrenamiento <- iris[iris.indices,]
> iris.test <- iris[-iris.indices,]
> # Modelo de regresión lineal multiple
> modelo.RLM<-lm(Sepal.Length ~ Petal.Length +
+                Sepal.Width + Petal.Width,
+                iris.entrenamiento)
> modelo.RLM

Call:
lm(formula = Sepal.Length ~ Petal.Length + Sepal.Width + Petal.Width,
    data = iris.entrenamiento)

Coefficients:
 (Intercept)  Petal.Length   Sepal.Width   Petal.Width
      1.8623        0.7404        0.6351       -0.6184

> summary(modelo.RLM)

Call:
lm(formula = Sepal.Length ~ Petal.Length + Sepal.Width + Petal.Width,
    data = iris.entrenamiento)

Residuals:
     Min       1Q   Median       3Q      Max
-0.83082 -0.22020  0.02456  0.20710  0.83794

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.86228    0.29770   6.256 9.63e-09 ***
Petal.Length  0.74045    0.06834  10.834  < 2e-16 ***
Sepal.Width   0.63509    0.08072   7.868 4.15e-12 ***
Petal.Width  -0.61835    0.15820  -3.909 0.000168 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3291 on 101 degrees of freedom
Multiple R-squared:  0.8623,    Adjusted R-squared:  0.8582
F-statistic: 210.7 on 3 and 101 DF,  p-value: < 2.2e-16

> # Análisis de los residuos del modelo
> residuos <- residuals(modelo.RLM)
> # Comprobación media de los residuos igual a cero
> mean(residuos)
[1] -1.107732e-17
> # Comprobación normalidad de los residuos
> lillie.test(residuos)

        Lilliefors (Kolmogorov-Smirnov) normality test

data:  residuos
D = 0.061604, p-value = 0.4226

> # Comprobación homocedasticidad
> bptest(modelo.RLM)

        studentized Breusch-Pagan test

data:  modelo.RLM
BP = 6.4722, df = 3, p-value = 0.09077
```

```
> # Comprobación covarianza término error y variable
> # explicativa es cero
> cov(residuos,iris.entrenamiento$Petal.Length)
[1] 3.091644e-17
> cor(residuos,iris.entrenamiento$Petal.Length)
[1] 5.295713e-17
> cov(residuos,iris.entrenamiento$Petal.Width)
[1] 2.709697e-17
> cor(residuos,iris.entrenamiento$Petal.Width)
[1] 1.097213e-16
> cov(residuos,iris.entrenamiento$Sepal.Width)
[1] 3.512466e-17
> cor(residuos,iris.entrenamiento$Sepal.Width)
[1] 2.452366e-16
> # Predicciones con datos de validacion (iris.test)
> prediciones <-predict(modelo.RLM, iris.test)
> # Error RMSE
> rmse(iris.test$Sepal.Length,prediciones)
[1] 0.2814553
```

Figura 2.23 Regresión lineal múltiple.

Regresión múltiple paso a paso

La regresión múltiple paso a paso es una optimización de la regresión lineal donde se calculan las variables significativas del modelo introduciendo y sacando variables, de forma que quedan seleccionadas solamente las que proporcionan una menor pérdida de información a la hora de explicar el modelo y penalizadas aquellos que presentan mayor número de parámetros.

Otros tipos de regresión

Otros algoritmos de regresión son la **regresión polinomial** donde la relación se realiza mediante una función polinomial de orden *n*, la **regresión lineal multivariante** que predice más de una variable respuesta a partir de variables independientes, la **regresión robusta** que predice utilizando una aproximación resistente al efecto de observaciones influyentes y, en general, la **regresión no lineal**, que realiza la predicción a través de modelos no lineales.

Mediante una transformación apropiada, se puede tratar de realizar un cambio de variable para linealizar los modelos.

Además, otros algoritmos vistos en el apartado de clasificación, con ligeras modificaciones, también pueden aplicarse como algoritmos de regresión para la predicción de valores continuos, las **máquinas de vector soporte aplicadas a regresión** (SVR), el método de **vecinos más cercanos** (*k*-NN), **árboles de decisión** (DTR), **bosques aleatorios** (RFR) y **redes neuronales artificiales** (RNA).

2.4.3 Algoritmos de detección de anomalías

La gran cantidad de datos disponibles actualmente en muchos sectores hace que se puedan utilizar algoritmos de aprendizaje no supervisado para analizar patrones buscando anomalías.

> *La **detección de anomalías** es una técnica que permite el reconocimiento de patrones en un conjunto de datos para identificar observaciones estadísticamente extrañas que no se comportan de la forma esperada en el resto de los datos.*

Generalmente, este tipo de observaciones anómalas puede tener origen o provocar algún tipo de problemas.

Algunas aplicaciones de detección de anomalías son las siguientes:

- Diagnóstico y monitorización de pacientes.
- Detección de fallos en procesos industriales y control de calidad.
- Fraude en tarjetas de crédito.
- Detección de intrusiones en la seguridad de redes de sistemas informáticos.

Los algoritmos desarrollados para la detección de anomalías se pueden clasificar en los siguientes tipos:

- Métodos basados en proximidad, como LOF y COF.
- Métodos fundamentados en agrupación como CBLOF e IForest.
- Métodos basados en densidad como DBSCAN.
- Algoritmos estadísticos como HBOS.
- Técnicas subespaciales como PCA.
- Otros algoritmos de clasificación: máquinas de vector soporte y redes neuronales.

El **factor anómalo local** (LOF) mide la desviación local de un punto respecto al grado en el cual un objeto depende de la densidad de sus vecinos locales. Asigna un valor a cada punto que representa la probabilidad de que ese dato sea una anomalía. Este puntaje es una comparación de densidades locales.

El **factor anómalo basado en la conectividad** (COF) es similar a LOF, pero realiza la estimación de la densidad local de su vecindario usando una técnica de camino más corto, que es denominada distancia de encadenamiento.

El **factor atípico local pasado en grupos** (CBLOF), a diferencia de los algoritmos anteriores que están basados en el método de vecinos más cercanos, utiliza técnicas de agrupación para establecer los grupos según la densidad en el conjunto de datos y posteriormente calcular un coeficiente de atipicidad para cada uno de los grupos que se han determinado, seleccionando como anómalos los datos con mayor coeficiente.

El método de **IForest**, *isolated forest*, está basado en el de *random forest*. Se construyen árboles realizando los puntos de división de forma aleatoria, hasta que cada observación queda aislada en un nodo terminal. Las observaciones anómalas quedan separadas al realizar un número pequeño de divisiones, que se usa como puntaje. Cuantas menos divisiones, mayor probabilidad tiene un elemento de ser una anomalía.

Los **métodos basados en densidad** consideran que un dato es anómalo cuando se encuentra en una zona de baja densidad de datos, es decir, apartado o con pocos elementos cercanos.

El algoritmo de **agrupamiento espacial basado en densidad de aplicaciones con ruido** (DBSCAN), *density-based scan clustering of application with noise*, establece que la densidad alrededor de un elemento está relacionada con el número de datos que tenga cada elemento dentro de un radio definido.

Las **técnicas basadas en aproximaciones estadísticas** utilizan modelos estadísticos creados a partir de los datos y evalúan la pertenencia de un objeto a dicho conjunto.

La **detección de anomalías basada en histogramas** (HBOS), *histogram-based outlier score*, se basa en la creación de un histograma para cada característica y la altura del bloque en el que se encuentra cada observación.

El **análisis de componentes principales** (PCA) utiliza una reducción de la dimensionalidad, pasando las observaciones a un espacio de menor número de características, pero conservando la mayor información posible. Los elementos anómalos serán mal trasladados al nuevo espacio y su reconstrucción en el espacio original será peor, por lo que se emplea el error de reconstrucción como criterio para identificar las anomalías.

Para saber más...

PyOD es un conjunto de herramientas de Python para detectar objetos atípicos en datos multivariados, que incluye más de 30 algoritmos.

2.4.4 Algoritmos de *clustering*

El **análisis de conglomerados** o ***clustering*** está formado por una serie de técnicas de aprendizaje no supervisado que permiten agrupar los datos en función de su similitud. Estos grupos se denominan ***clusters***.

A continuación, se van a desarrollar el método jerárquico y el método no jerárquico de *k*-medias.

Análisis jerárquico

El **análisis jerárquico,** *hierarchical clustering,* es un grupo de métodos de agrupación no supervisado en los que en cada fase del proceso se realizan una serie de particiones o fusiones sucesivas. Es capaz de determinar por sí mismo el número de grupos.

La representación jerárquica del grupo se realiza con un diagrama llamado **dendograma**.

Existen dos tipos de algoritmos jerárquicos: aglomerativo y divisivo. El primero es el más utilizado de los dos y es el más sencillo de programar debido a que solo existe una forma de juntar dos grupos y, sin embargo, hay muchas maneras de dividirlos.

- **Algoritmo divisivo:** se trata de un agrupamiento de arriba hacia abajo partiendo de un único *cluster* con todas las observaciones que se va dividiendo recursivamente en grupos menos similares hasta que haya uno por dato.
- **Algoritmo aglomerativo:** parte de cada observación aislada y consiste en combinar repetidamente los dos grupos que se encuentran más cerca en otro más grande. Se calcula la distancia entre cada par de datos y se almacena en una matriz de distancias.

  A continuación, se asigna cada observación a un grupo y se unen los pares de datos más cercanos según la información almacenada en la matriz de distancias, disminuyendo la cantidad de grupos en una unidad.

  Se repite el proceso hasta que todos los grupos se unan en un solo *cluster*.

Ejemplo 20

En la Figura 2.24 se muestra el código para agrupar las especies del conjunto de datos iris mediante el algoritmo de agrupamiento jerárquico aglomerativo junto con el dendograma.

```
> library(MASS)
> data(iris)
> set.seed(123)
> datos.iris <- iris
> # Estandarización de los datos
> datos.iris[,1:4] <- scale(datos.iris[,1:4])
> # Matriz de distancias
> matriz.distancias <- dist(datos.iris[,1:4])
> # modelo jerárquico (hierachical clustering)
> modelo.hc <- hclust(matriz.distancias)
> modelo.hc

Call:
hclust(d = matriz.distancias)

Cluster method   : complete
Distance         : euclidean
Number of objects: 150

> # dendograma
> plot(modelo.hc,hang = -1,cex=0.6,labels = datos.iris[,5], main= "dendograma")
> rect.hclust(modelo.hc, k=3, border="red")
```

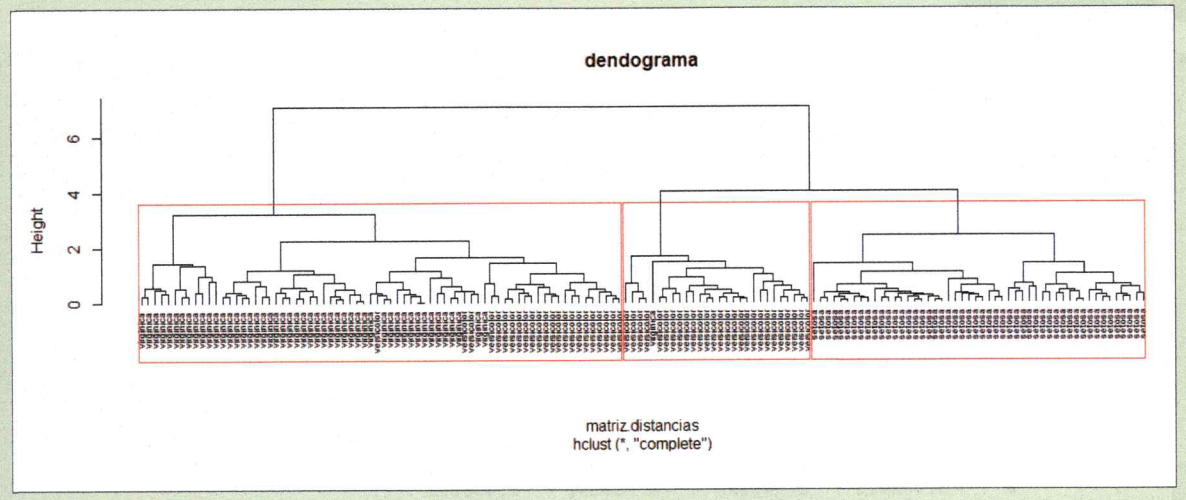


Figura 2.24 Agrupamiento jerárquico.

Es un algoritmo sensible a los valores atípicos, a la medida de distancia elegida y las variables tienen que estar estandarizadas. Necesita una alta carga computacional, por lo que no es recomendable cuando el conjunto de observaciones es grande.

K-medias

***K*-medias,** o *k-mean,* es un algoritmo de agrupamiento no supervisado que busca formar *k* grupos a partir de un conjunto formado por *n* observaciones, basándose en las características de los datos.

Es necesario establecer previamente el número de *cluster k* debido a que el método no es capaz de determinarlo por sí mismo.

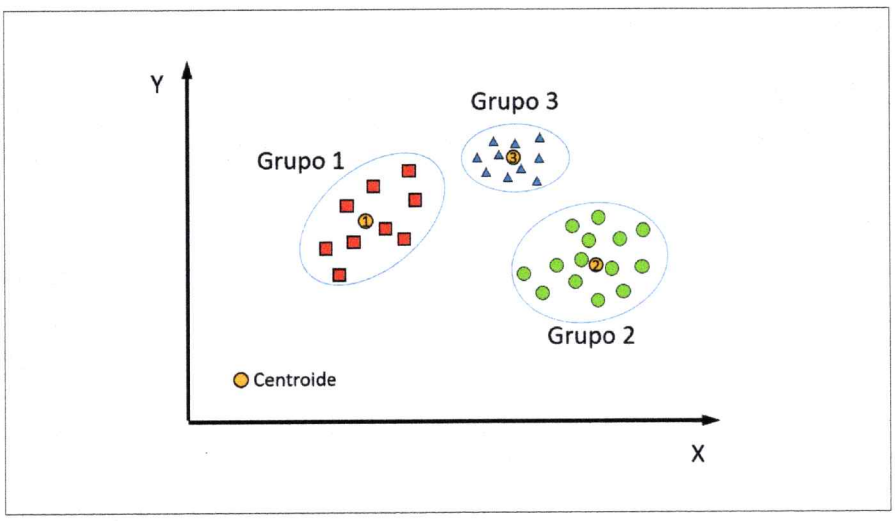


Figura 2.25
K-medias.

El agrupamiento se realiza resolviendo un problema de optimización en el que se minimiza la distancia cuadrática de cada dato al centroide de su grupo. Se utiliza el criterio de la inercia, que considera que el modelo está optimizado cuando los grupos tengan una distancia máxima entre ellos (inter-*cluster*) y una distancia mínima entre los miembros del grupo (intra-*cluster*).

El algoritmo determina en primer lugar los *k* centroides en el espacio de los datos, asigna a cada dato el centroide más cercano y actualiza la posición de cada centroide, sucesivamente.

Ejemplo 21

En la Figura 2.26 se muestra el código para agrupar las especies del conjunto de datos iris mediante el algoritmo de *clustering* de *K*-medias para $k = 3$. Dado que en este caso se dispone de un conjunto de datos etiquetados, se puede calcular la precisión que ha alcanzado el algoritmo en la clasificación.

```
> library(MASS)
> data(iris)
> set.seed(123)
> library(stats)
> # Modelo K-means con k=3
> modelo.K_means <- kmeans(iris[,1:4], centers = 3)
> modelo.K_means
K-means clustering with 3 clusters of sizes 50, 62, 38

Cluster means:
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1     5.006000    3.428000     1.462000    0.246000
2     5.901613    2.748387     4.393548    1.433871
3     6.850000    3.073684     5.742105    2.071053

Clustering vector:
  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 [35] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 [69] 2 2 2 2 2 2 2 2 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 2
[103] 3 3 3 3 2 3 3 3 3 3 3 2 2 3 3 3 2 3 2 3 2 3 3 2 2 3 3 3 3 2 3 3
[137] 3 3 2 3 3 3 2 3 3 3 2 3 3 2

Within cluster sum of squares by cluster:
[1] 15.15100 39.82097 23.87947
 (between_SS / total_SS =  88.4 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
> tabla <- table(modelo.K_means$cluster, iris[,5])
> tabla

   setosa versicolor virginica
1      50          0         0
2       0         48        14
3       0          2        36
> # Cálculo del porcentaje de clasificación correcta
> 100 * sum(diag(tabla)) / sum(tabla)
[1] 89.33333
```

Figura 2.26 Agrupamiento con algoritmo *K*-medias.

2.4.5 Algoritmos de refuerzo del aprendizaje

A lo largo de los apartados anteriores se ha visto que el aprendizaje no supervisado busca relaciones o patrones y el aprendizaje supervisado trata de decidir qué es algo.

El **aprendizaje por refuerzo,** o *reinforcement learning*, consiste en la enseñanza a una computadora a hacer algo.

De esta forma, aplicando este tipo de aprendizaje, una computadora puede aprender a conducir un coche, circular por un laberinto complejo o a jugar a juegos como el ajedrez.

El **agente** es el programa informático que está aprendiendo. El objetivo del agente tiene que estar definido, ya que todas sus acciones van dirigidas a lograr este objetivo.

Se va a enfrentar a situaciones diversas y al actuar, recibe recompensas o castigos por sus acciones. Tomando como base estos premios, puede ir optimizando su comportamiento para lograr maximizar las recompensas.

El **entorno** es un conjunto de características llamadas variables de estado, como pueden ser la ubicación, la velocidad o la distancia al objetivo. El conjunto de valores numéricos en un momento dado *t* constituye el estado de entorno en ese momento.

El entrenamiento consiste en el proceso de aprendizaje del agente a partir de la experiencia. En cada instante *t*, el agente va a realizar una acción que está condicionada por la recompensa que va a recibir, pero el objetivo no debe ser obtener la mejor recompensa a corto plazo, sino la que le reporte el mayor beneficio en todo el episodio. Por ejemplo, en el juego del ajedrez, el agente puede optar por comer una pieza de menor valor entre las posibles o, incluso no comer una pieza, si le aporta una ventaja a largo plazo, ya que el objetivo del juego es ganar, no comer el mayor número de fichas.

La **política ε-voraz** o avara, *ε-greedy policy*, es la política que va a decidir las acciones que realizará el agente. Consiste en llevar a cabo la mejor acción en base a la información disponible en cada momento. No obstante, en determinadas ocasiones, para introducir la posibilidad de exploración y que el agente pueda no tomar la mejor decisión disponible, de vez en cuando, podrá adoptar una acción al azar con una probabilidad. Así se permite explorar todas las opciones posibles.

La **velocidad de aprendizaje** α determina cuánto se aprende de cada experiencia y el **factor de descuento** Υ indica la importancia del largo plazo. Ambos parámetros deben ser ajustados y pueden tomar valores entre 0 y 1. Cuanto mayor sea Υ, más elevada será la visión, pero se pueden tener mayores problemas de convergencia. Con valores de $\alpha = 0$ no aprenderá y $\alpha = 1$, se actualiza siempre y olvida todo lo aprendido.

La **función de valor** *Q* es aquella encargada de estimar la recompensa que recibirá el agente desde un estado *t* hasta el final del episodio.

El **algoritmo *Q-Learning*** emplea una estrategia para elegir la acción con una política *ε-greedy*, con parámetros Υ y α .

La característica más importante de este algoritmo es que es capaz de aprender la función *Q*-óptima a partir de cualquier experiencia disponible para ese entorno. Asume que se elige la mejor acción y que, por tanto, la función de valor de la forma que se consiga la máxima recompensa.

El algoritmo comienza inicializando la función de valor a cero o a valores aleatorios, para cada par de estado-acción y se va creando una tabla en la que cada par de estado (s) y acción (a) tiene asociada el valor que devuelve la función de valor $Q(s, a)$, repitiendo hasta convergencia.

Por tanto, si para un estado existen dos acciones posibles, a_1 y a_2, la función de valor calculará $Q(s, a_1)$ y $Q(s, a_2)$. Si, por ejemplo, $Q(s, a_1) = 3$ y $Q(s, a_2) = 5$, el agente realizará la acción a_2, ya que considera que es mejor pues le aporta mayor recompensa.

2.4.6 Otros algoritmos relacionados con el aprendizaje automático

A continuación, se estudian diversos algoritmos relacionados con el aprendizaje automático.

Reducción de la dimensión

Los dos algoritmos más conocidos de la reducción de la dimensionalidad lineal son el análisis de componentes principales y el análisis factorial.

- **Análisis de componentes principales** (PCA): es una técnica que se utiliza para reducir la dimensión de grandes conjuntos de datos, con el objetivo de que la pérdida de información ocasionada sea mínima. Esto se consigue creando unas nuevas variables que no presenten correlación entre sí, o que sea lo más pequeña posible, y que maximiza la varianza sucesivamente. El problema para encontrar estas variables nuevas se reduce al cálculo de los autovalores y autovectores.

Los vectores propios o autovectores van a originar unos nuevos ejes de coordenadas que siguen las direcciones en los que hay mayor variabilidad en el conjunto de datos, así el que tiene mayor valor propio o autovalor es el de la dirección de mayor dispersión y el que tenga un valor propio bajo el de menor dispersión.

Las componentes principales no tienen un significado sobre el conjunto de datos a reducir la dimensionalidad, ya que están construidas en base a combinaciones lineales de las variables de dicho conjunto inicial.

Las etapas de un análisis de componentes principales son las siguientes:

- Estandarizar las variables iniciales, si fuese necesario.
- Hallar la matriz de covarianza para conocer las correlaciones.
- Calcular los valores propios y los vectores propios para poder identificar las componentes principales.
- Crear un vector de características y así decidir las componentes principales a conservar.
- Transformar los datos a los ejes de las componentes principales.

Ejemplo 22

En la Figura 2.27 se muestra el código para obtener las componentes principales PC1 y PC2 del conjunto de datos iris.

```
> library(MASS)
> data(iris)
> set.seed(123)
> # Se elimina la variable dependiente
> datos.iris <- iris[-5]
> # Modelo PCA
> modelo.PCA <- prcomp(datos.iris)
> modelo.PCA
Standard deviations (1, .., p=4):
[1] 2.0562689 0.4926162 0.2796596 0.1543862

Rotation (n x k) = (4 x 4):
                    PC1         PC2         PC3         PC4
Sepal.Length  0.36138659 -0.65658877  0.58202985  0.3154872
Sepal.Width  -0.08452251 -0.73016143 -0.59791083 -0.3197231
Petal.Length  0.85667061  0.17337266 -0.07623608 -0.4798390
Petal.Width   0.35828920  0.07548102 -0.54583143  0.7536574
> summary(modelo.PCA)
Importance of components:
                          PC1     PC2    PC3     PC4
Standard deviation     2.0563 0.49262 0.2797 0.15439
Proportion of Variance 0.9246 0.05307 0.0171 0.00521
Cumulative Proportion  0.9246 0.97769 0.9948 1.00000
> # Guardar PC1 y PC2
> comp_princp<-modelo.PCA$x[,1:2]
> str(comp_princp)
 num [1:150, 1:2] -2.68 -2.71 -2.89 -2.75 -2.73 ...
 - attr(*, "dimnames")=List of 2
  ..$ : NULL
  ..$ : chr [1:2] "PC1" "PC2"
```

Figura 2.27 Análisis de componentes principales.

- **Análisis factorial:** es una técnica que busca la relación entre las variables de un conjunto de datos mediante otra serie de menor número de variables latentes que no son observables, que se llaman factores, y que tengan sentido y sean interpretables. Se construye un modelo de factores específicos y factores comunes.

  Sus etapas son las siguientes:

  - Estandarizar las variables iniciales, si fuese necesario.
  - Calcular la matriz de covarianza para conocer las correlaciones.

- Extraer el número óptimo de factores.
- Realizar la rotación de la solución para favorecer su interpretación.
- Estimar los elementos en las nuevas dimensiones.

La diferencia principal entre las técnicas de análisis factorial y de componentes principales radica en que en la primera se distingue entre la varianza común (compartida con otras variables) y única (de la propia variable) mientras que en la segunda no hay esta distinción, solo analiza la varianza total.

Algoritmos de aprendizaje profundo

Los **algoritmos de aprendizaje profundo,** o *deep learning,* se implementan mediante redes neuronales profundas tales como redes neuronales convolucionales, recurrentes, generativas antagónicas o *autoencoders* que se estudiarán en la Unidad 5.

2.5 Procedimientos del *machine learning*: datos, identificación de patrones y toma de decisiones

El proceso de implementación de un modelo de aprendizaje automático para resolver un problema y poder tomar una decisión tiene una serie de pasos que se indican a continuación:

- **Definición del problema:** en primer lugar, se debe comprender el problema para poder analizarlo. Esto significa determinar cuál es el objetivo del estudio, si se disponen de los datos y conocimientos necesarios para poder abordarlo y cómo se van a medir los resultados. Es decir, qué se va a hacer, cómo se va a llevar a cabo y si es posible ejecutarlo.
- **Definición de un criterio de evaluación:** para poder determinar si una solución es correcta o cuánto se aparta de la solución esperada es necesario determinar un criterio de evaluación. Este criterio no será otro que la medida del error. Existen muchos criterios de evaluación o medidas del error establecidos. Por ejemplo, en problemas de regresión es habitual utilizar el error cuadrático medio (RMSE) o *root-mean square error* y para problemas de clasificación la entropía cruzada.
- **Recopilación y preparación de los datos:** el siguiente paso es la recopilación de datos, que pueden estar fácilmente disponibles o no, ser internos a la organización o externos y, en este último caso, gratuitos o de pago.

  Una vez recopilados los datos se deben preprocesar para hacer una limpieza de los datos que no aportan valor, comprobar que la matriz de datos está correctamente estructurada y con el formato adecuado, tratar el problema de los datos incompletos y atípicos, determinar si existen correlaciones entre las diferentes características, realizar el normalizado de los datos si fuese necesario y dividir los datos en los subconjuntos de entrenamiento y validación.
- **Elección del modelo:** elegir la técnica de aprendizaje automático que sea adecuada a la tarea que se plantea. Para ello se utilizan los algoritmos de agrupamiento, clasificación, regresión, aprendizaje profundo. También dependerá del tipo de datos que se pretende procesar, es decir, sin son numéricos, categóricos, texto, imágenes, audio o vídeo.
- **Construcción del modelo:** una vez elegido, se realiza la fase de aprendizaje del modelo con los datos del conjunto de entrenamiento. A continuación, se evalúa el modelo con los datos del conjunto de validación en función del criterio de evaluación que se ha establecido previamente.
- **Análisis de errores:** en esta etapa se analizan los errores para determinar si el modelo elegido es válido. Si es necesario, se ajustan los parámetros

mediante un proceso de ensayo y error o, incluso, cambiando el algoritmo por otro que se adapte mejor al problema planteado.

- **Optimización del modelo:** se pueden aplicar técnicas para mejorar el modelo establecido y disminuir el error que se comete en la validación o también emplear estrategias para reducir la carga computacional del modelo.

  Una vez que el error está dentro de los límites establecidos, y debidamente generalizado, evitando el sobreajuste, se puede aplicar a la resolución del problema con datos diferentes a los de los conjuntos de entrenamiento y validación.

  De esta forma se han identificado los patrones, establecido relaciones y determinado resultados para poder tomar decisiones basadas en datos.

2.6 Herramientas de aprendizaje automático

Algunas de las herramientas, servicios y lenguajes de programación más conocidos y utilizados son los que se describen a continuación.

Para recordar...

La CPU es la unidad central de procesamiento y la GPU es la unidad de procesamiento gráfico.

Microsoft Azure ML Studio: Microsoft ha creado una plataforma para desarrollar modelos de aprendizaje automático en la nube con una interfaz altamente visual y que no necesita de conocimientos previos. Así, se puede realizar tanto el modelo como la etapa de entrenamiento rápidamente y con una relativa sencillez, posibilitando, además, comunicar y compartir los resultados entre miembros de una organización. Trabaja con CPU y GPU.

Amazon Machine Learning (AML): proporciona un servicio de aprendizaje automático multipropósito en la nube desarrollado a partir de la misma tecnología que han usado internamente en sus investigaciones de datos. Es capaz de implementar datos de varias fuentes como RedShit, RDS y Amazon S3. Este servicio permite desarrollar modelos para tareas de clasificación y de regresión. Los conjuntos de datos están limitados a un tamaño máximo de 100 Gb y no se permite la exportación de los modelos obtenidos.

El servicio únicamente está disponible para los clientes que tienen una cuenta de suscripción, y no acepta nuevos usuarios ya que está proporcionando el servicio SageMaker.

Watson Machine Learning: es una oferta de IBM en la nube que permite incorporar herramientas visuales de inteligencia artificial para detectar patrones y hacer más versátil el proceso de toma de decisiones mediante la utilización de una plataforma dinámica que permite una mayor facilidad en el intercambio de información y comunicación entre los miembros del equipo de trabajo. También permite la gestión e implementación de módulos avanzados para aprendizaje profundo utilizando modelos de redes neuronales.

Google Cloud Machine Learning Engine: es una plataforma que presenta una gran similitud a Amazon Sagemaker. Alojado en la nube, se utiliza para algoritmos complejos, siendo capaz de manejar conjuntos de entrenamiento con capacidad de hasta miles de millones de datos con una alta velocidad. Tiene una gran ventaja, ya que permite la posibilidad de utilizar otras herramientas para entrenar los modelos y después implementarlos, puesto que no está restringido solo a los entrenados dentro de Google.

Es usado para aplicaciones como la detección de nubes en imágenes tomadas por satélites.

El precio por los servicios de esta plataforma es sustancialmente inferior al resto de proveedores según informa Google.

BigML: es una plataforma integral de aprendizaje automático en la nube con el objetivo de hacerlo accesible a todo el mundo en cuanto a precio, programable y escalable, presentando poca dificultad técnica y bajo nivel de formación necesario para manejarlo.

Es capaz de importar los datos de varias fuentes, construir los modelos rápidamente e insertarlos en aplicaciones móviles para emplearlos en realizar predicciones.

Esta plataforma alberga una gran cantidad de algoritmos para aprendizaje supervisado que se pueden utilizar para tareas de regresión, clasificación y predicción de series temporales, así como de aprendizaje no supervisado.

Es utilizado por multinacionales como Mazda y Avast, y Telefónica ha entrado a participar en su capital.

Hasta una cantidad de 16 MB hay un plan gratuito. A partir de dicha cantidad de datos, los planes premium de pago empiezan con precios bajos para fomentar su accesibilidad, aumentando según las necesidades.

Dataiku: es un servicio que permite la interacción y colaboración entre los usuarios y analistas de tal forma que pueden trabajar conjuntamente.

Incluye gran cantidad de bibliotecas actualizadas permitiendo la utilización de los lenguajes de programación Python y R.

Knime: es una aplicación de software libre y código abierto basada en GUI, fácil de entender ya que no requiere de conocimientos de programación, para análisis y generación de informes, por lo que es utilizada en diversos sectores como el de la industria farmacéutica y también en servicios, tanto de análisis de inteligencia empresarial como en el análisis de datos financieros.

Utiliza un entorno sencillo, visual e intuitivo que integra los lenguajes de programación Python, R, Java, JavaScript, C y C++.

Google Tensorflow: en un principio esta plataforma fue desarrollada por Google para su propio uso interno, pero posteriormente fue puesta a disposición del público como código abierto, pudiendo ser ejecutada en la nube y también de forma local por todos los sistemas operativos y dispositivos, procesados por CPU o GPU.

Es utilizado por compañías como Dropbox, Twitter y eBay. Entre las muchas aplicaciones donde se utiliza, se puede reseñar el análisis de radiografías para diagnóstico médico y la separación mediante software de la imagen de una persona del fondo, cuando anteriormente para realizar esta operación era necesario tomar la fotografía con dos cámaras.

Weka *(Waikato Environment for Knowledge Analysis):* es una biblioteca que contiene algoritmos de aprendizaje automático de código abierto escrita en Java, accesible a través de una interfaz gráfica. Está muy extendida en el ámbito de la docencia y la investigación debido a la sencillez de su funcionamiento.

Contiene algoritmos para el preprocesamiento de los datos, tareas de regresión, clasificación, agrupamiento, reglas de asociación, visualización de datos, así como también en problemas de aprendizaje profundo.

PyTorch: es una herramienta de aprendizaje profundo gratuita fácil de usar, programada en Python. Incorpora de forma nativa la ejecución en GPU, lo que le confiere gran rapidez en los cálculos. Ha absorbido al marco de aprendizaje profundo Caffe2.

Es utilizado por Facebook como librería principal para las aplicaciones de aprendizaje profundo.

RapidMiner: es una aplicación de aprendizaje automático a través de un entorno visual, que tiene la capacidad de implementar diversos modelos al problema que se presenta e identificar el que mejor se adapta. Además, como no son necesarios grandes conocimientos de programación es bastante sencilla de aprender a manejar.

Cuenta con dos componentes: RapidMiner Studio para realizar las tareas de aprendizaje automático y RapidMiner Server que permite el trabajo entre múltiples usuarios y entornos como Python y R.

Es utilizada por compañías como Nokia, Cisco, Ford, Honda, E.ON, IBM, Hewlett Packard.

Scikit-learn: se trata de una biblioteca gratuita para aprendizaje automático construida sobre las librerías de Python: NumPy, Matplotlib y SciPy. Incorpora modelos para preprocesamiento, reducción de la dimensión, regresión, clasificación y agrupamiento, muy adecuada para principiantes ya que cuenta con una gran documentación.

Keras: es un *software* de código abierto que proporciona una interfaz en Python para el uso de redes neuronales y aprendizaje profundo, diseñado para ir construyendo la arquitectura de cada red neuronal por bloques, permitiendo la aceleración de cálculo al utilizar los recursos de la GPU.

Keras se emplea en centros como el CERN y la NASA.

Amazon SageMaker: es un servicio que ha sido creado por Amazon con el objetivo de ofrecer el desarrollo de modelos de aprendizaje automático a empresas para la realización de predicciones en sus sectores de negocio, cobrando unas tasas en función de los recursos de procesamiento y computación requeridos.

Rstudio: al igual que RCommander, es una interfaz para usar R, que es un lenguaje de programación para computación estadística que, a través de múltiples librerías, permite implementar algoritmos de aprendizaje automático. Es uno de los *software* más populares, pero requiere conocimientos de programación avanzados.

Jupyter Notebook: es una aplicación web cliente-servidor de código abierto compatible con varios lenguajes de programación. El nombre elegido para esta aplicación rinde un homenaje a tres lenguajes de programación: Julia, Python y R. *Jupyter Notebook* se emplea para escribir la sintaxis del código y también para ejecutarlo, visualizando los resultados que se obtienen.

Accord.net: es una herramienta que incluye paquetes para procesamiento de imagen y audio para entrenar modelos en lenguaje C#, siendo su aplicación más utilizada en contenido de visión artificial y en la manipulación de archivos de audio.

Python: es un lenguaje de programación multiplataforma de código abierto no compilado que se puede emplear en el desarrollo de aplicaciones muy diversas, pero sobre todo es muy utilizado en tareas de aprendizaje automático.

GitHub: es uno de los portales con mayor número de recursos dentro de la red, a través del cual se puede alojar en esta plataforma código de forma gratuita. Debido a que es bastante fácil de utilizar, es muy usada para colaborar entre los desarrolladores de código.

Tableau: es una aplicación diseñada para la inteligencia de negocios usada por grandes multinacionales como Barclays, Deloitte o Tesla Motors. Su misión es preprocesar grandes cantidades de datos, analizarlos y compartirlos, en una plataforma amigable y de forma rápida y sencilla, ayudando a ver y comprender la información, cuya programación es en R.

Power Bi: Microsoft Power Bi es un conjunto de herramientas de análisis empresarial diseñado para extraer los datos de múltiples orígenes, combinarlos, realizar un análisis y presentarlos a través de gráficos e informes interactivos de una forma sencilla.

Colab: es un entorno interactivo de Google en la nube que permite escribir y ejecutar el código de Python mediante un notebook similar a Jupyter, con la ventaja del uso de máquinas virtuales con acceso gratuito a GPUs.

Julia: es un lenguaje de programación de código abierto, cuya ventaja fundamental sobre otros lenguajes de programación como R o Python es la velocidad debido a que es un lenguaje compilado y a que ha sido diseñado para la computación paralela y distribuida.

Anaconda: es una suite que integra más de 150 paquetes orientados al aprendizaje automático y la ciencia de datos, desde donde se pueden crear desarrollos para Python y R. No se debe confundir con Conda, que es el gestor de paquetes y los entornos virtuales.

H2O: es una plataforma de aprendizaje automático y profundo de código abierto escrito en Java. Es posible acceder a todas sus funciones desde R o Python.

2.7 Aplicaciones del *machine learning*

El aprendizaje automático se ha convertido en una herramienta muy utilizada por las empresas debido a su capacidad creciente para identificar patrones y realizar predicciones de comportamientos:

- Análisis de imágenes, audio y vídeo.
- Predicciones meteorológicas, de episodios de contaminación y de tráfico en las ciudades.
- Identificación de clientes potenciales, análisis del comportamiento del consumidor, sistemas de recomendaciones.
- Vehículos autónomos y robots.
- Clasificación de *spam* y detección de *software* malicioso como antivirus.
- Detección del rostro para desbloquear móviles y filtros en apps.
- Detección de fraudes en transacciones económicas, reducción del riesgo en la concesión de créditos y detección de señales de impago en el sector bancario.
- Prevenir y combatir ciberataques.
- Programar rutas por empresas de transporte para la reducción de tiempos de entrega.
- Diagnósticos médicos en radiografías, detección de cáncer y otras patologías.
- Motores de búsqueda online.
- Predicción de fallos en maquinaria y defectos de calidad en el sector industrial.
- Juegos y videojuegos para interactuar con *bots*.
- En el sector del turismo para realizar predicciones que ayuden a agencias de viajes, aerolíneas y hoteles.

Resumen

- El **aprendizaje automático supervisado** parte de un conjunto de datos de entrenamiento que están previamente etiquetados. El **aprendizaje automático no supervisado** parte de un conjunto de datos sin etiquetar. El **aprendizaje semisupervisado** es un caso mixto.
- Los algoritmos más empleados en tareas de clasificación son el análisis discriminante, los *K* vecinos más próximos, los árboles de decisión como CART, C5.0 y *random forest*, el clasificador bayesiano ingenuo o *Naïve-Bayes*, la regresión logística, las máquinas de soporte vector y las redes neuronales artificiales.
- La **regresión lineal** es una técnica de represión que busca la relación lineal entre unas variables independientes de entrada y una variable dependiente de salida.
- Las **redes neuronales artificiales** están formadas por neuronas situadas en una capa de entrada, una o varias capas ocultas y una capa de salida. Se utilizan para tareas de clasificación y de regresión.
- El **análisis de conglomerados** o ***clustering*** está formado por una serie de técnicas de aprendizaje no supervisado que permiten agrupar los datos en función de su similitud.
- El **aprendizaje por refuerzo** consiste en la enseñanza a una computadora a hacer algo, mediante un sistema de recompensas y penalizaciones.
- El **procedimiento del aprendizaje automático** consta de los siguientes pasos: definición del problema y de un criterio de evaluación, recopilación y preparación de los datos, elección del modelo y su construcción, análisis de errores y optimización del modelo. Una vez validado se puede emplear para determinar patrones, predecir datos y tomar decisiones.

Test de evaluación

1. Según la definición de aprendizaje automático planteada por Tom Mitchell, ¿cuál es la tarea T de una aplicación de correo electrónico que aprende a filtrar mejor el correo no deseado al observar qué correos marca usted como no deseados?
   a) Observar cómo usted etiqueta los correos electrónicos como correo deseado o correo no deseado.
   b) La cantidad de correos electrónicos correctamente clasificados como deseados y no deseados.
   c) Clasificar los correos electrónicos como deseados o no deseados.
   d) Ninguna de las anteriores, pues no se trata de un algoritmo de aprendizaje automático.

2. Elija la respuesta correcta de las siguientes afirmaciones:
   a) Los algoritmos k-NN y k-means son algoritmos de clasificación supervisados.
   b) Los algoritmos k-NN y k-means son algoritmos de clasificación no supervisados.
   c) El algoritmo k-NN es de clasificación supervisado y k-means no es supervisado.
   d) El algoritmo *k-NN* es no supervisado y *k-means* es supervisado.

3. ¿Cuál es la diferencia entre el aprendizaje supervisado y no supervisado?
   a) El aprendizaje supervisado utiliza solo datos de entrenamiento y el no supervisado utiliza datos de entrenamiento, validación y prueba.
   b) El aprendizaje supervisado utiliza datos no etiquetados en el entrenamiento y el no supervisado emplea datos etiquetados.
   c) El aprendizaje supervisado utiliza datos etiquetados en el entrenamiento y el no supervisado emplea datos no etiquetados.
   d) El aprendizaje supervisado necesita, al menos, un operario que introduzca los datos en el ordenador, mientras que el no supervisado no necesita ninguno.

4. Señale la afirmación o afirmaciones correctas:
   a) Las redes neuronales artificiales solo pueden emplearse para tareas de clasificación.
   b) Las máquinas de vector soporte se utilizan para tareas de clasificación y de regresión.
   c) El algoritmo de componentes principales se usa para reducir el número de variables del problema.
   d) Todas las anteriores son correctas.

5. Indique cuál de las siguientes afirmaciones es una definición razonable de aprendizaje automático:
   a) El aprendizaje automático es la ciencia de la programación de ordenadores.
   b) El aprendizaje automático es el campo que permite a los robots actuar de forma inteligente.
   c) El aprendizaje automático es el campo de estudio que permite a los ordenadores aprender sin ser programados explícitamente.
   d) El aprendizaje automático es la capacidad de algunos robots de aprender rápidamente.

6. Suponga que ha estado apuntado datos como humedad, temperatura y estado del día (bueno, malo) en una estación meteorológica que tiene en su casa durante un año y quiere predecir la temperatura y el tiempo que hará mañana a partir de los datos de su archivo. ¿Está ante un problema de qué tipo?
   a) Clasificación.
   b) Regresión.
   c) Clasificación y regresión.
   d) Ninguno de los anteriores, es de otro tipo.

Actividades

ACTIVIDAD 1

Realizar el Ejemplo 17 utilizando el 50% de los datos como conjunto de datos de entrenamiento y empleando una y tres neuronas en la capa oculta. Comparar los resultados obtenidos de la matriz de confusión y el porcentaje de aciertos en la clasificación.

ACTIVIDAD 2

Desarrollar el Ejemplo 18 para buscar la relación entre la anchura del pétalo y del sépalo en la especie setosa y de la longitud del pétalo y del sépalo de la especie versicolor utilizando un algoritmo de regresión lineal.

ACTIVIDAD 3

Crear un esquema de los métodos de aprendizaje automático siguiendo la siguiente estructura: aprendizaje supervisado: algoritmos de clasificación y algoritmos de regresión y aprendizaje no supervisado: algoritmos de *clustering*.

ACTIVIDAD 4

Mediante el algoritmo *k*-NN, generar la clasificación del conjunto de datos iris, con un conjunto de datos de entrenamiento del 80 % y el resto para realizar una predicción acerca del tipo de especie.